

第 3 章 計算機算術

Computer Arithmetic

Outline

CPU = CU + ALU + Reg.....

控制單元
算術邏輯單元

- ◆ Addition and subtraction
- ◆ Constructing an arithmetic logic unit
 - Building ALU
 - Add, sub, and, or, nor
 - Set-on-less-than, overflow detection, zero detection
 - Fast adders
 - Cascaded carry look-ahead adder
- ◆ Multiplication
 - Unsigned multiply
 - Signed multiply
- ◆ Division
- ◆ Floating point
 - Representations
 - Addition and multiplication

Slt

判斷ALU運算結果是否為0

前瞻進位加法器

Problem: Designing MIPS ALU

- ◆ Requirements: must support the following arithmetic and logic operations
 - **add, sub**: two's complement adder/subtractor with overflow detection (二補數加減法, overflow 偵測)
 - **and, or, nor**: logical AND, logical OR, logical NOR
 - **slt** (set on less than): two's complement adder with inverter, check sign bit of result 加法器 & Inverter (負S3)

slt \$s1, \$s2, \$s3

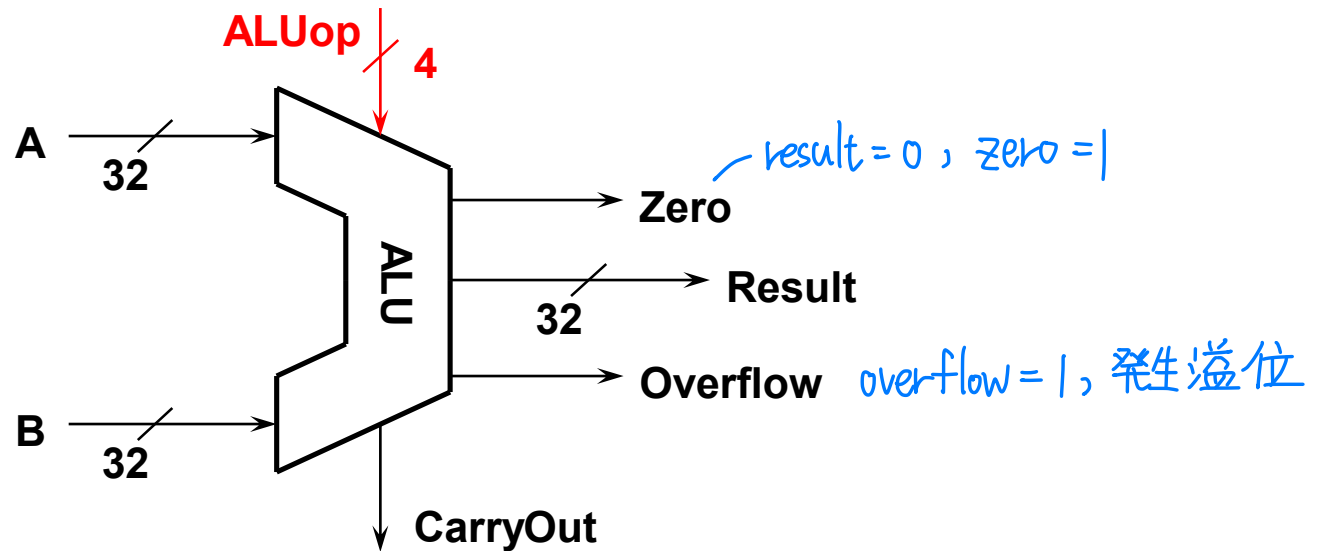
if $\$s2 < \$s3$, $\$s1 = 0$

$S2 - S3$?

$S2 + \underbrace{(-S3)}$

↓
S3取補+1

Functional Specification



ALU Control (ALUop)

0000

0001

0010

0110

0111

1100

Function

and

or

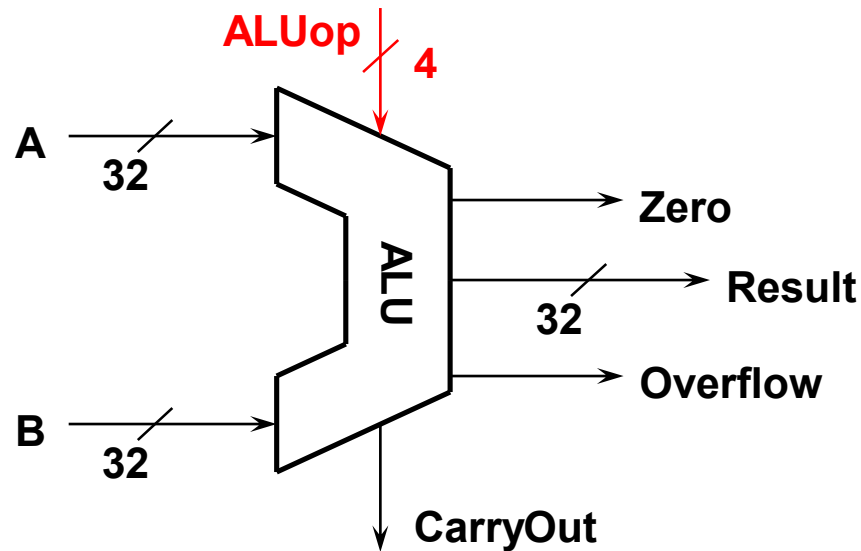
add

subtract

set-on-less-than

nor

Functional Specification



ALU Control (ALUop)

0000

0001

0010

0110

0111

1100

Function

and

or

add

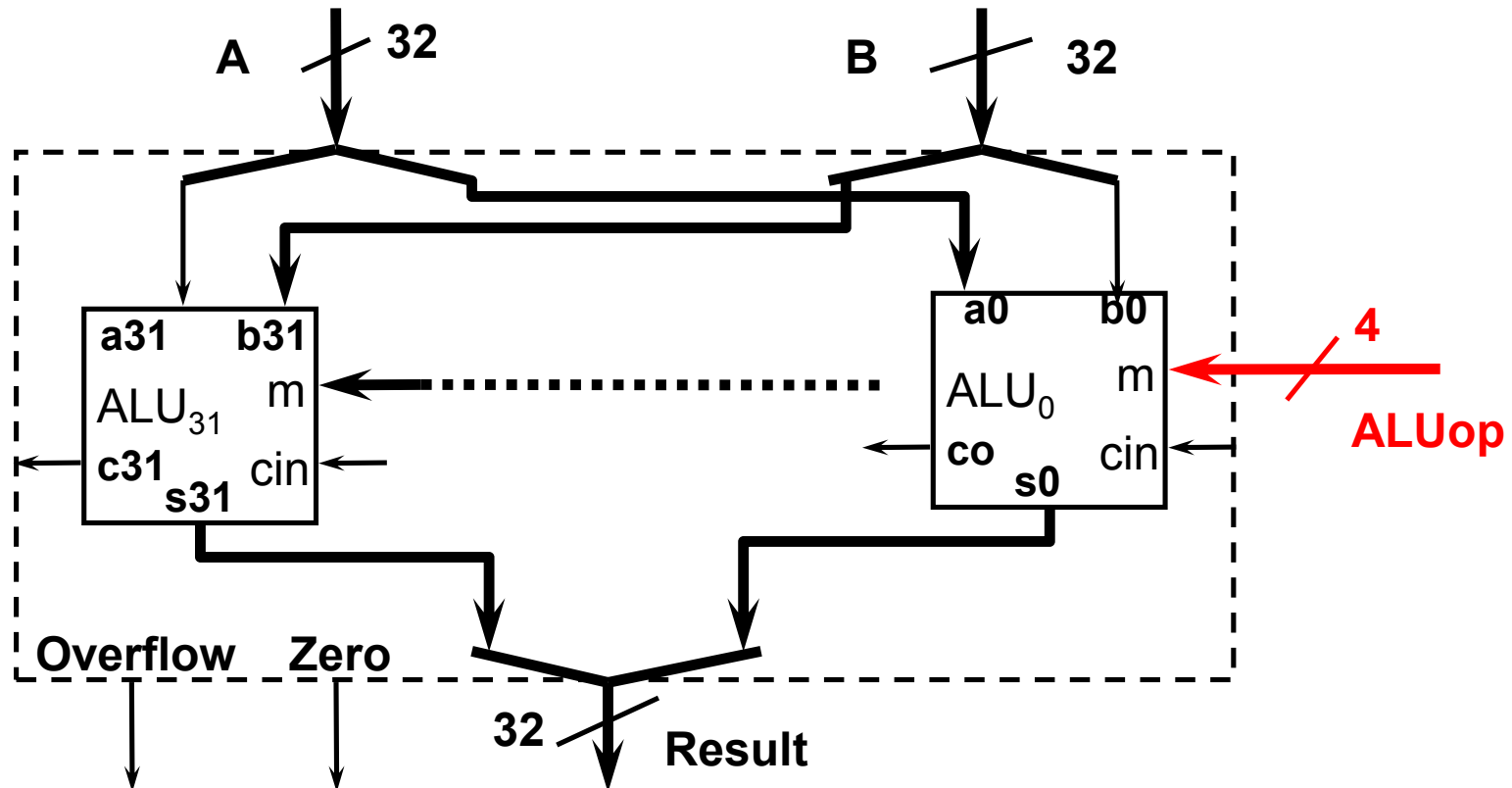
subtract

set-on-less-than

nor

A Bit-slice ALU

- ◆ Design trick 1: divide and conquer
 - Break the problem into simpler problems, solve them and glue together the solution
- ◆ Design trick 2: solve part of the problem and extend

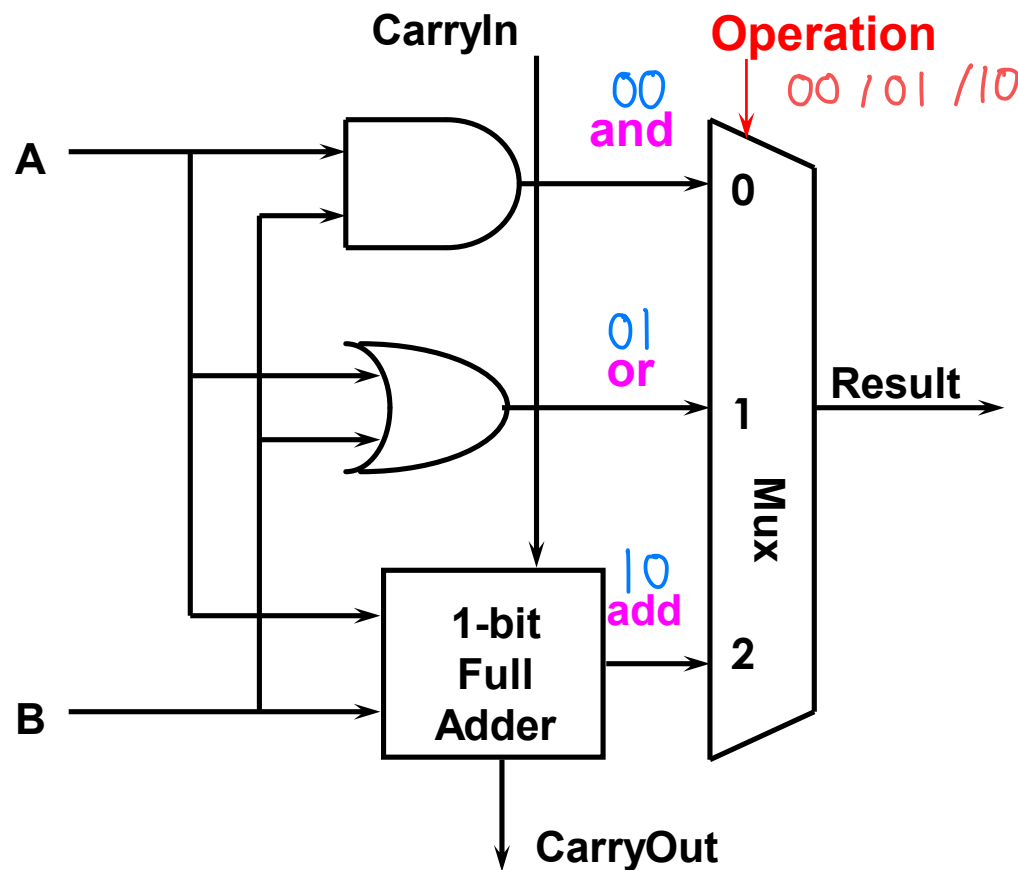


A 1-bit ALU

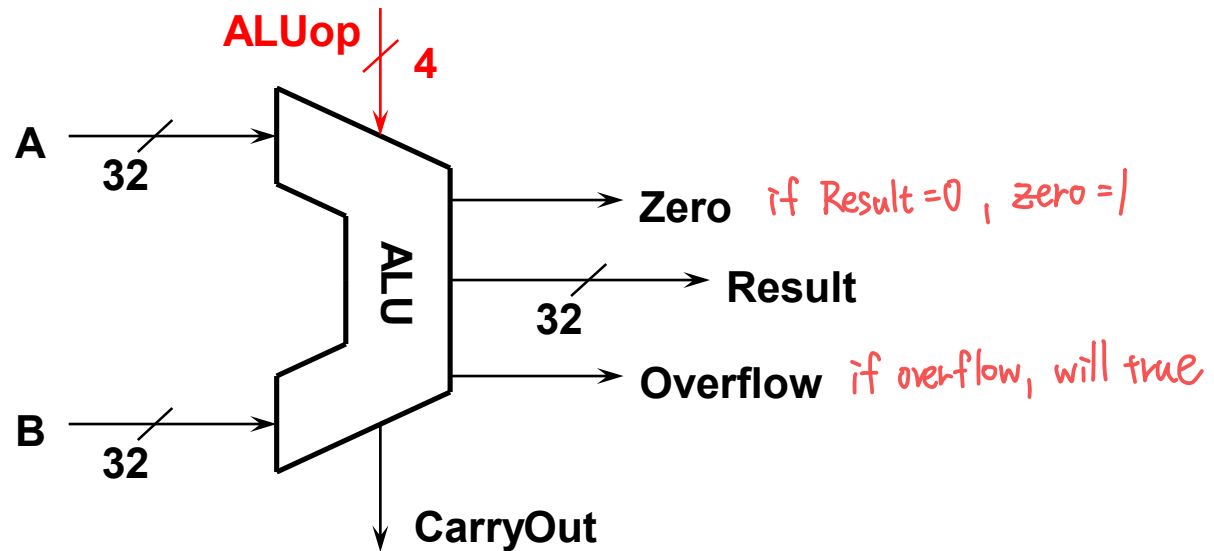
- ◆ Design trick 3: take pieces you know (or can imagine) and try to put them together

* 先做一 Bit

再把他黏成 32 个 Bit



Functional Specification



ALU Control (ALUop)

決定A、B
是否要反向

00	00
00	01
00	10
01	10
01	11
11	00

→ OPERATION!

Function

and

or

add

subtract

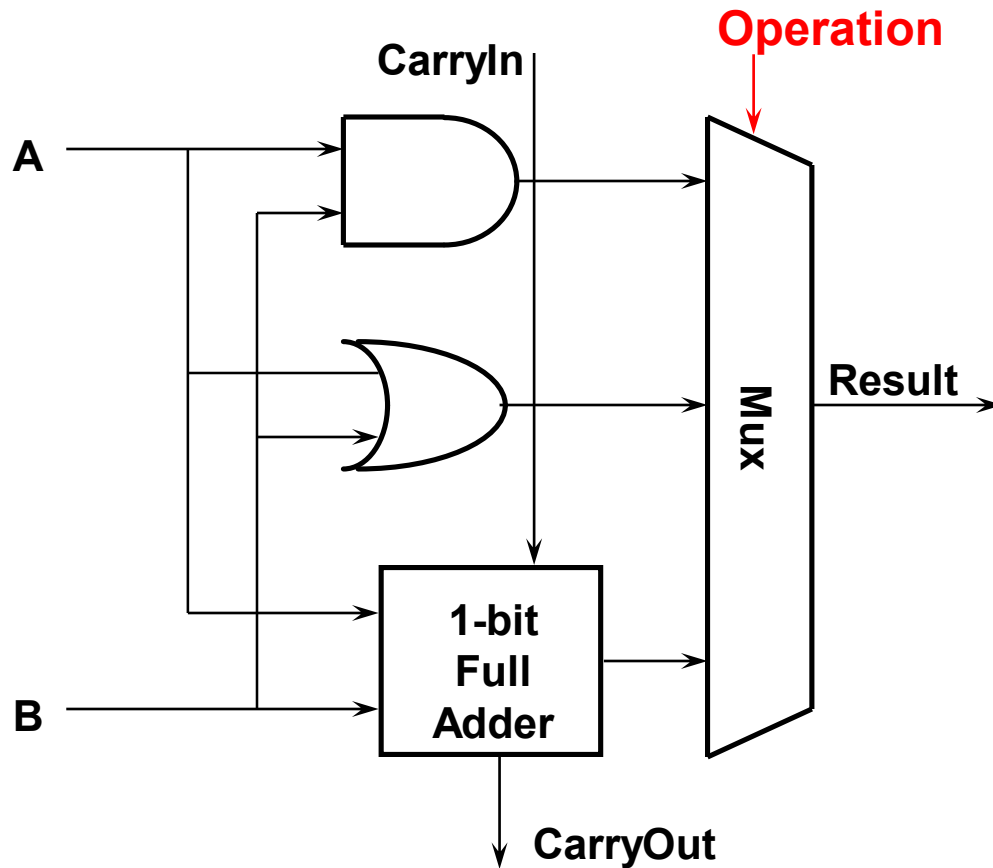
set-on-less-than

nor

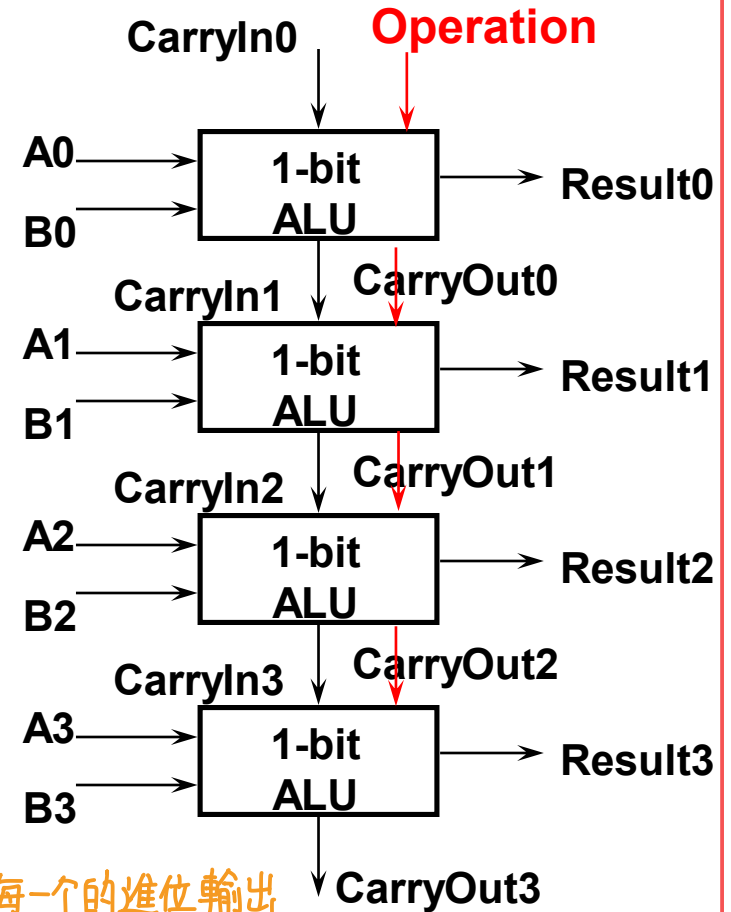
A 4-bit ALU

x4 → 16 Bit ALU

1-bit ALU



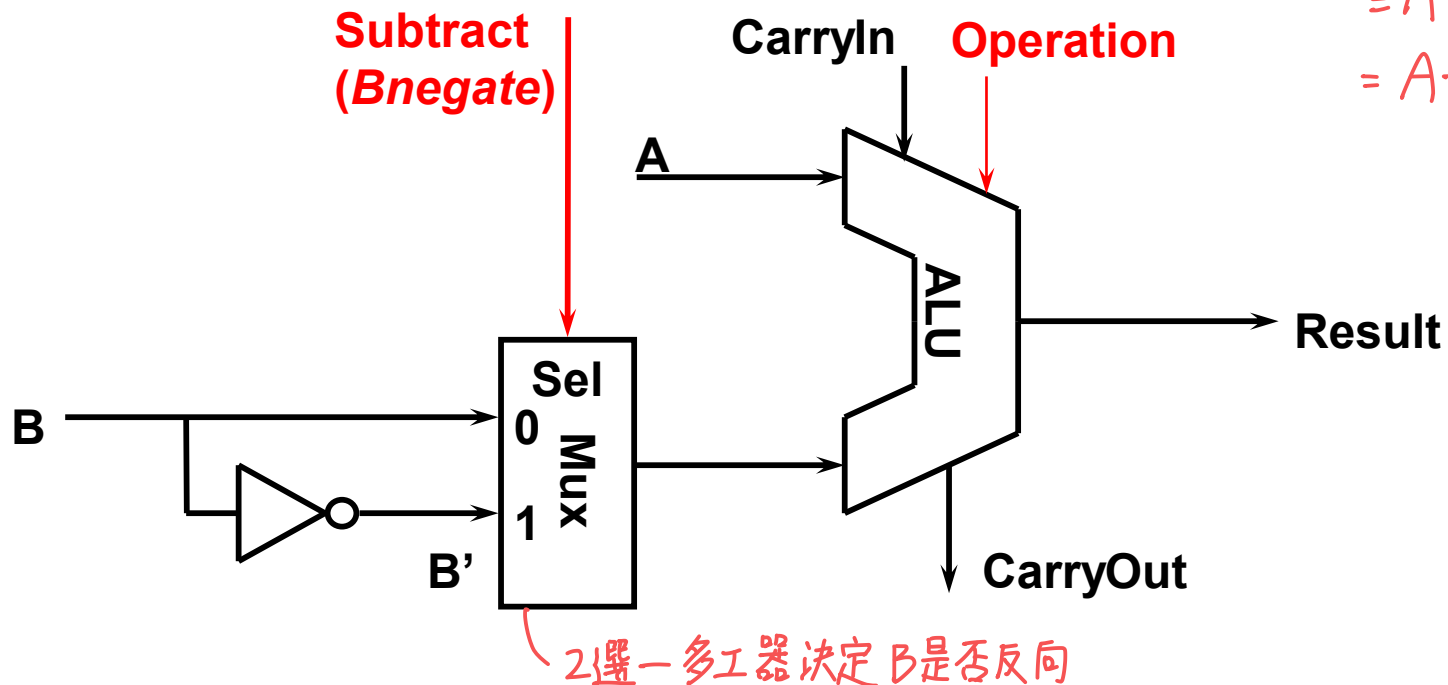
4-bit ALU



每一个的进位输出
接到下一个的进位输入

How about Subtraction?

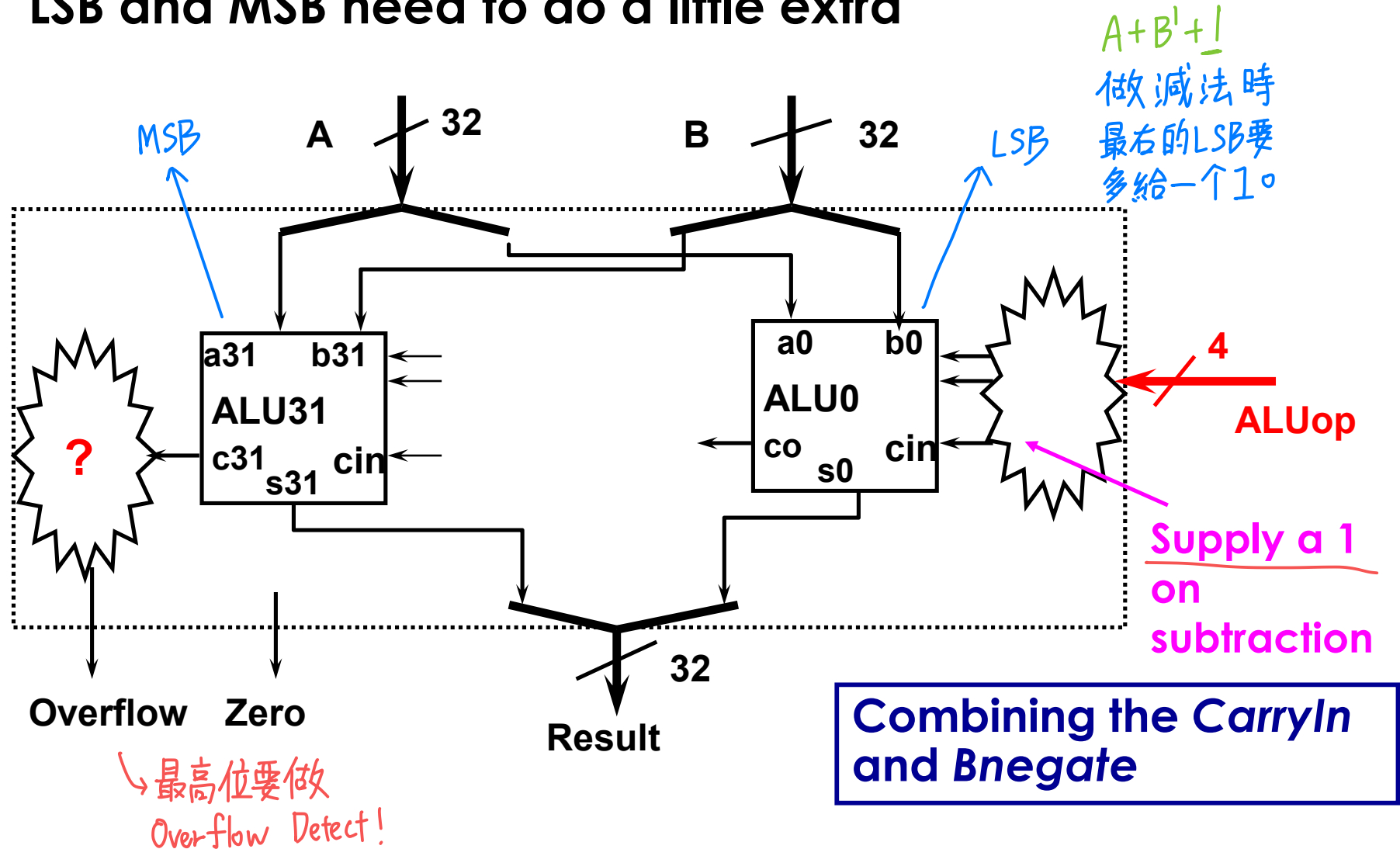
- ◆ 2's complement: take inverse of every bit and add 1 (at c_{in} of first stage)
 - $A + B' + 1 = A + (B' + 1) = A + (-B) = A - B$
 - Bit-wise inverse of B is B'



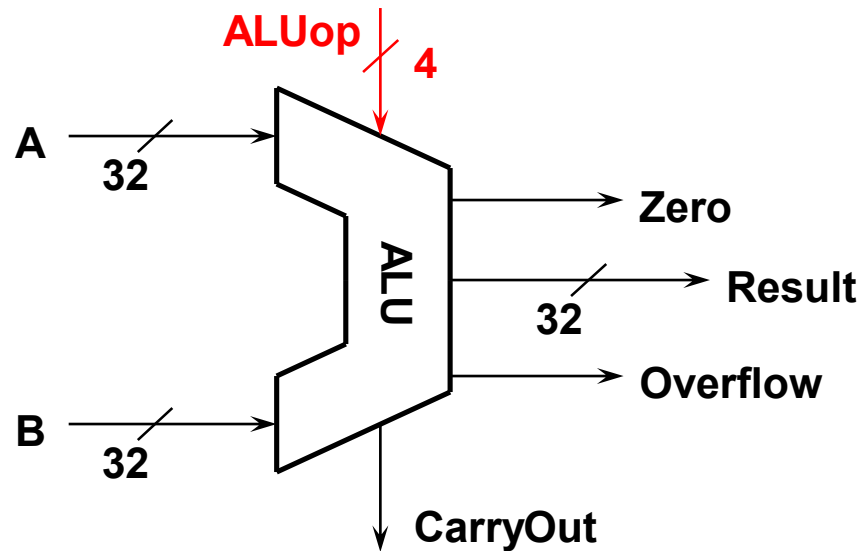
$$\begin{aligned} A - B \\ &= A + (-B) \\ &= A + B' + 1 \end{aligned}$$

Revised Diagram

- ◆ LSB and MSB need to do a little extra



Functional Specification



ALU Control (ALUop)

0000

0001

0010

0110

0111

1100

Function

and

or

add

subtract

set-on-less-than

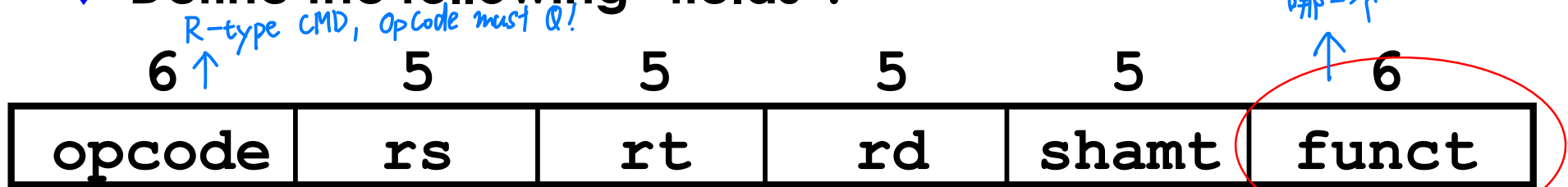
nor

控制 B
是否要反向!

← 01 10 → 減法時後兩位必定 10
(Control ALU Output Full Adder Result)

R-Format Instructions (1/2)

◆ Define the following “fields”:



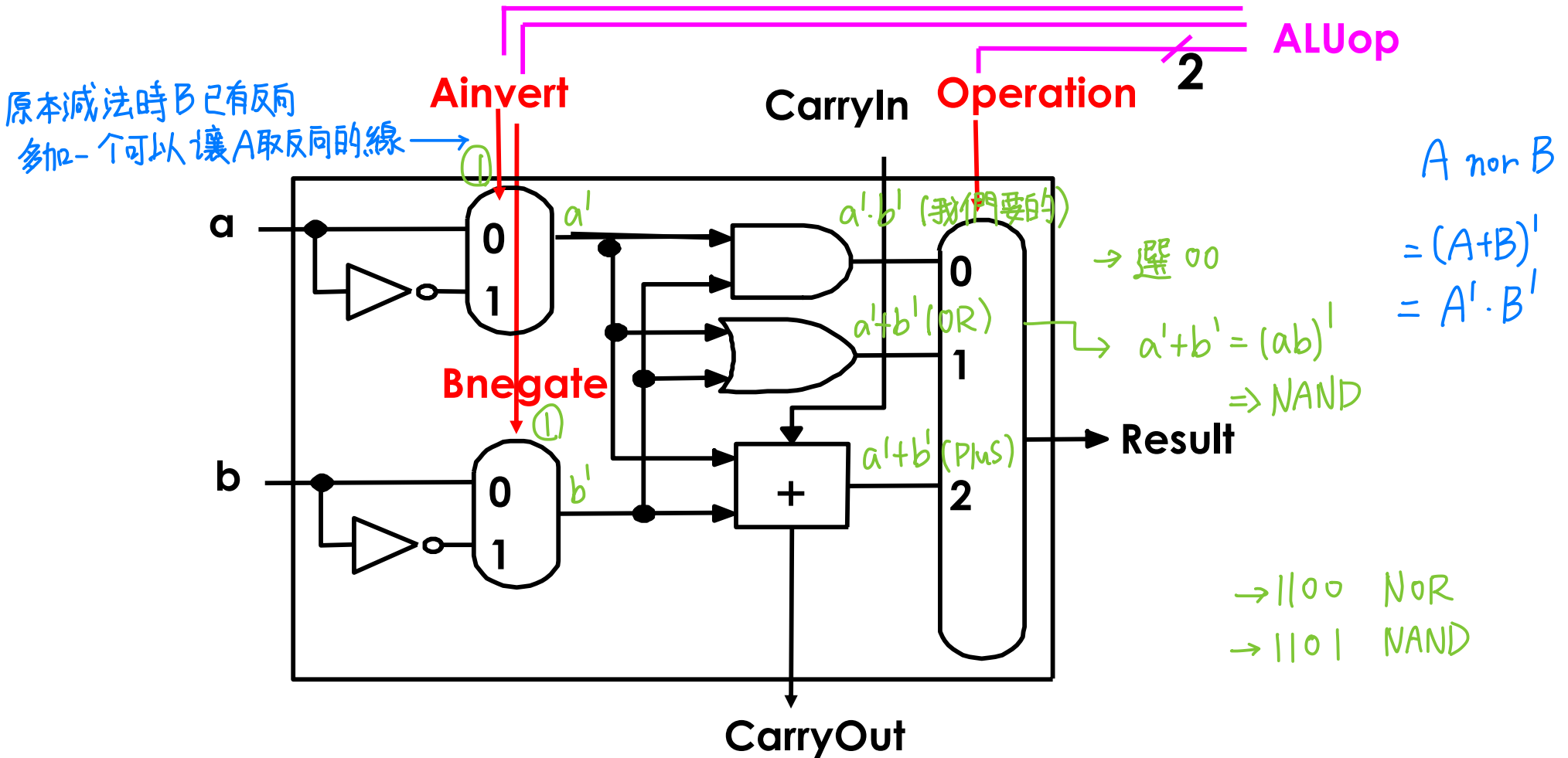
All R-Type CMD

- add
- sub
- and
- or
- nor
- slt
- sl
- srl

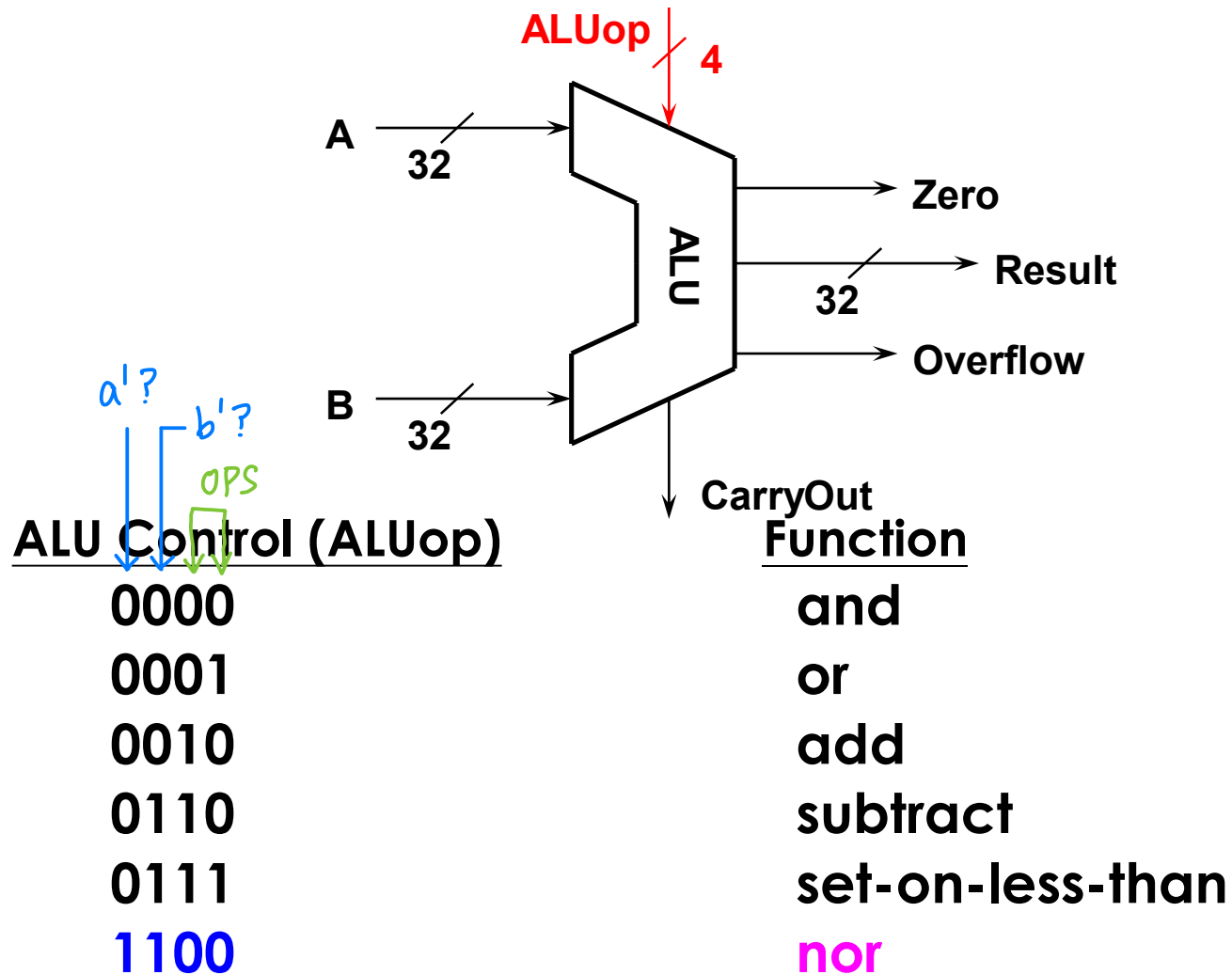
- opcode: partially specifies what instruction it is (Note: 0 for all R-Format instructions)
- funct: combined with opcode to specify the instruction
Question: Why aren't opcode and funct a single 12-bit field?
- rs (Source Register): generally used to specify register containing first operand
- rt (Target Register): generally used to specify register containing second operand
- rd (Destination Register): generally used to specify register which will receive result of computation

Nor Operation

◆ $A \text{ nor } B = (\text{not } A) \text{ and } (\text{not } B)$



Functional Specification



Outline

- ◆ Addition and subtraction
- ◆ Constructing an arithmetic logic unit
 - Building ALU
 - Add, sub, and, or, nor
 - Set-on-less-than, overflow detection, zero detection
 - Fast adders $\rightarrow$ `slt $S1 $S2 $S3 if ($S2 < $S3) $S1=1, else $S1=0`
 - Cascaded carry look-ahead adder

- ◆ Multiplication

- Unsigned multiply
 - Signed multiply

- ◆ Division

- ◆ Floating point

- Representations
 - Addition and multiplication

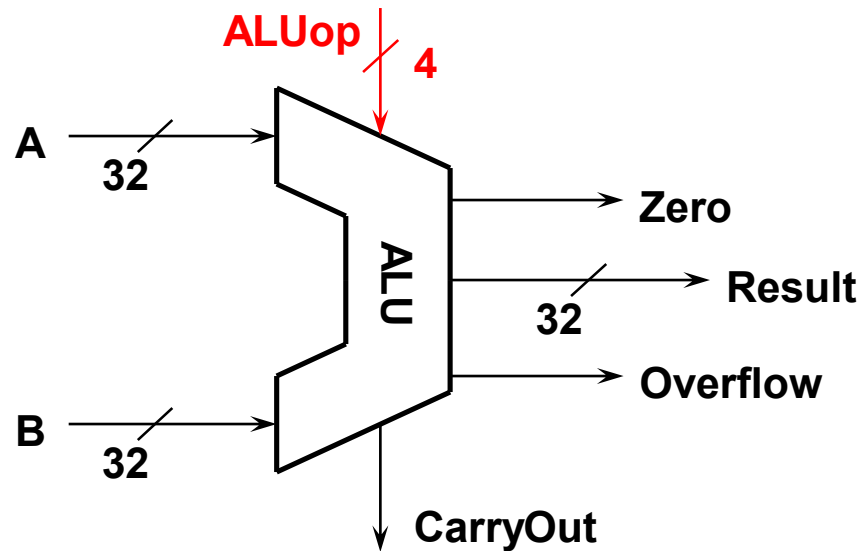
小於, 用相減做
假設相減完, MSB 是 1
就是小於

S2 0101 (5) 0111 (7)
S3 - 0111 (7) - 0101 (5)

1110 0010

↓
S1 的 1 怎麼來? 把這個丟給 S1? (注意! 丟給 S1 的最低位元)

Functional Specification



Slot \$S_1, S_2, S_3\$
 if \$S_2 < S_3\$ 32个Bit
 \$S_1 = \overbrace{0000 \dots 0001}\$

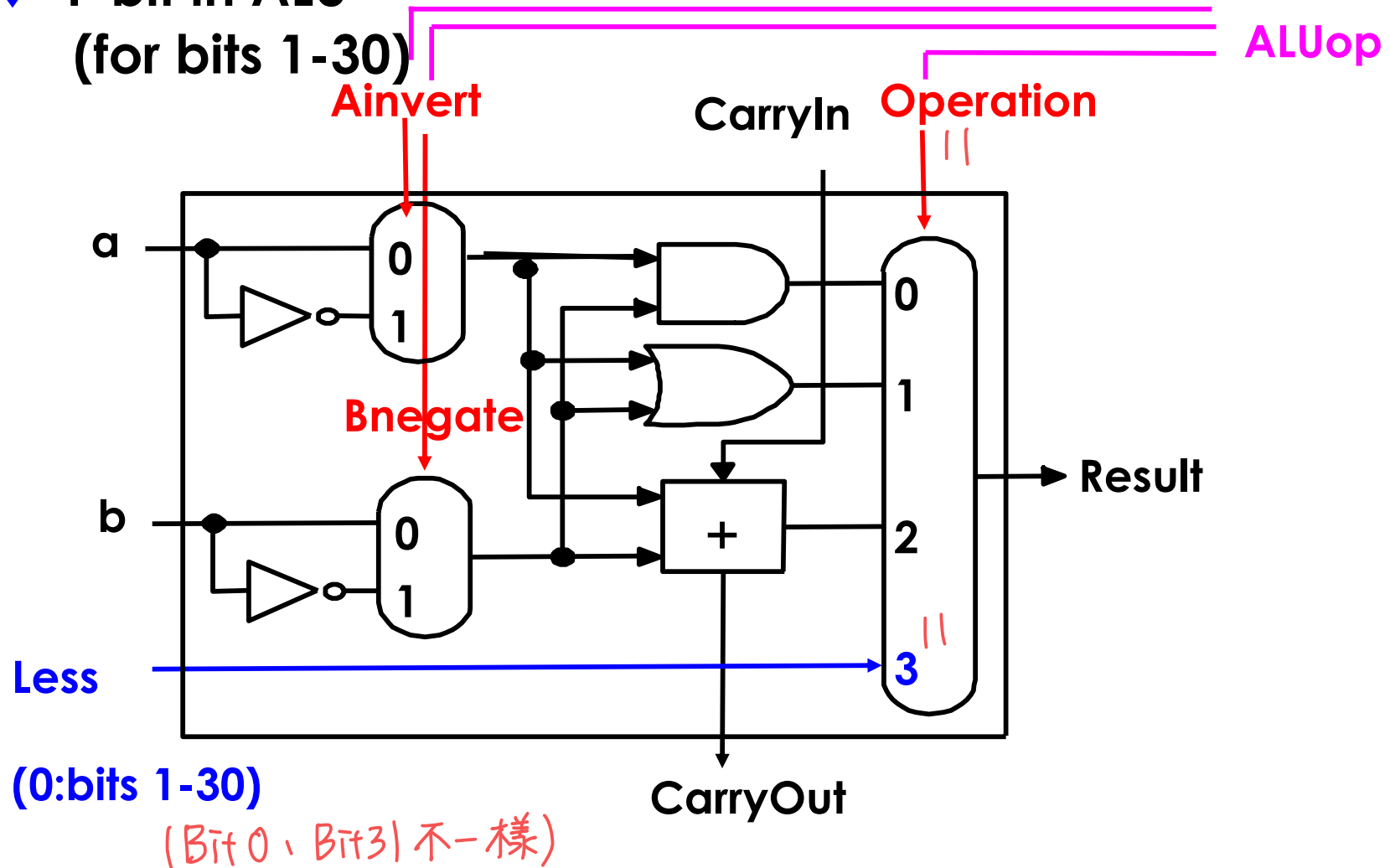
S2 x x x ... x
S3 - y y y ... y

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32

<u>ALU Control (ALUop)</u>	<u>Function</u>
0000	and
0001	or
0010	add
0110	subtract
0111	set-on-less-than
1100	nor

Set on Less Than (I)

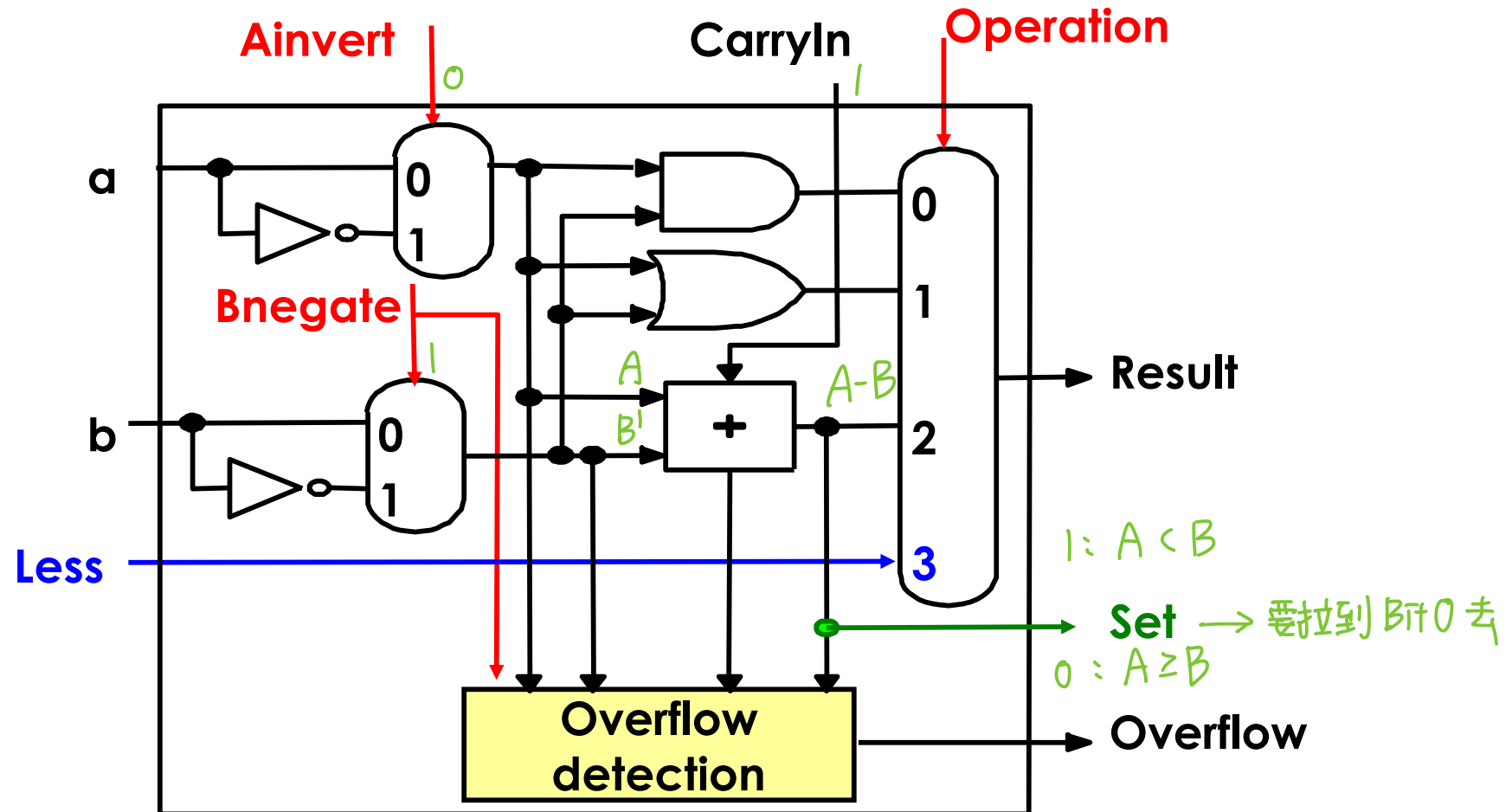
◆ 1-bit in ALU (for bits 1-30)



Set on Less Than (II)

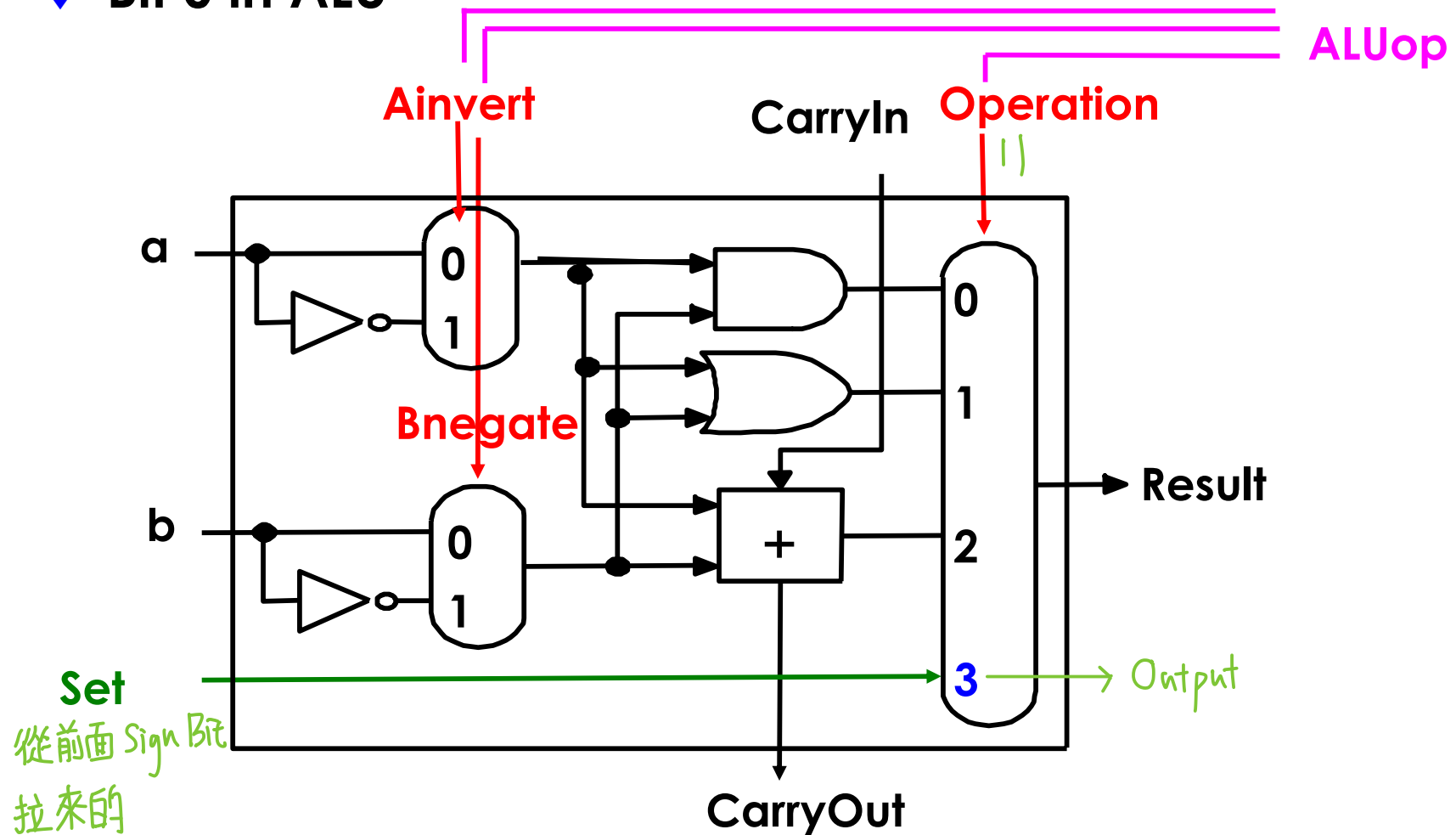
Bit 31

◆ Sign bit in ALU

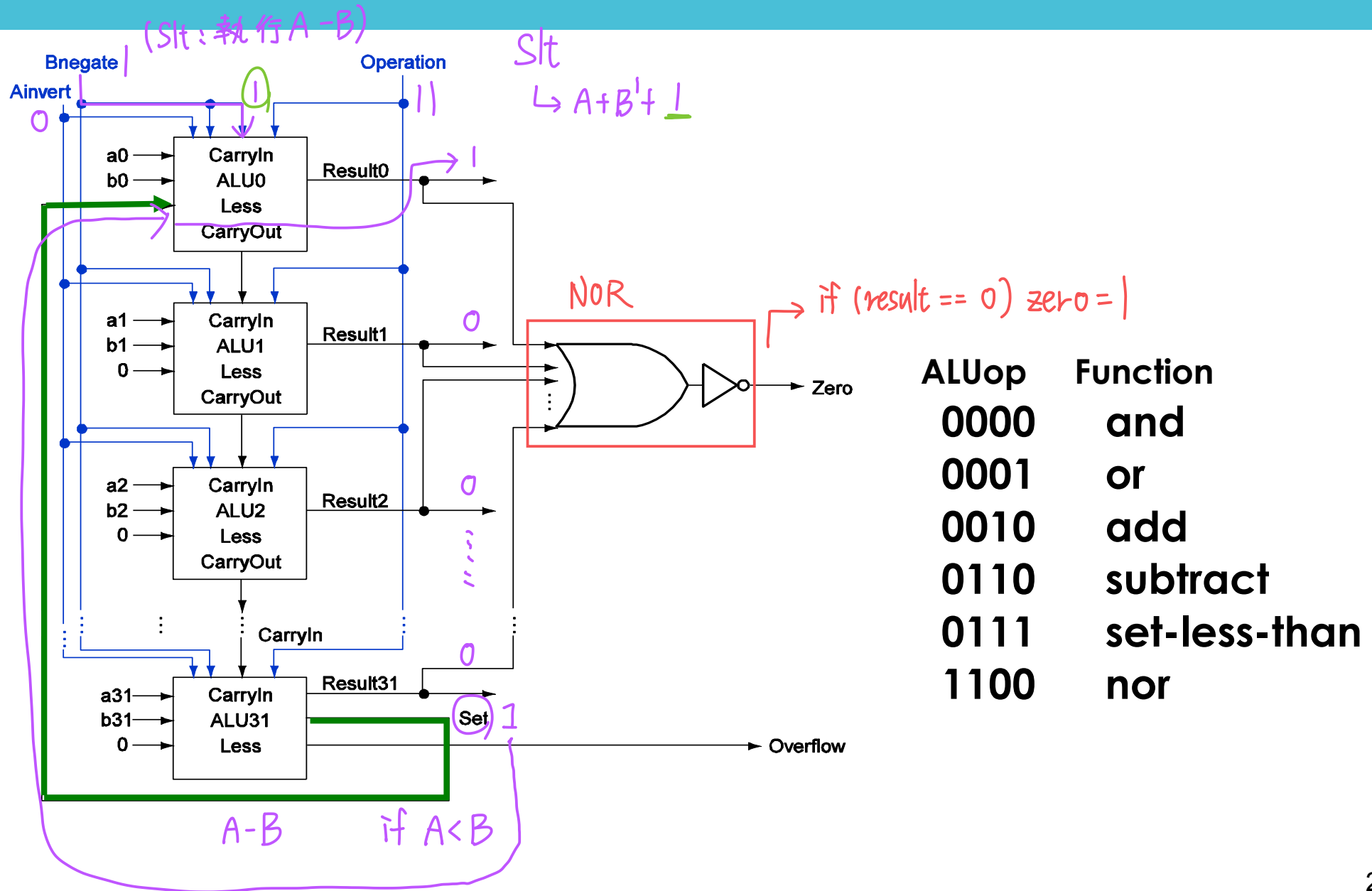


Set on Less Than (III)

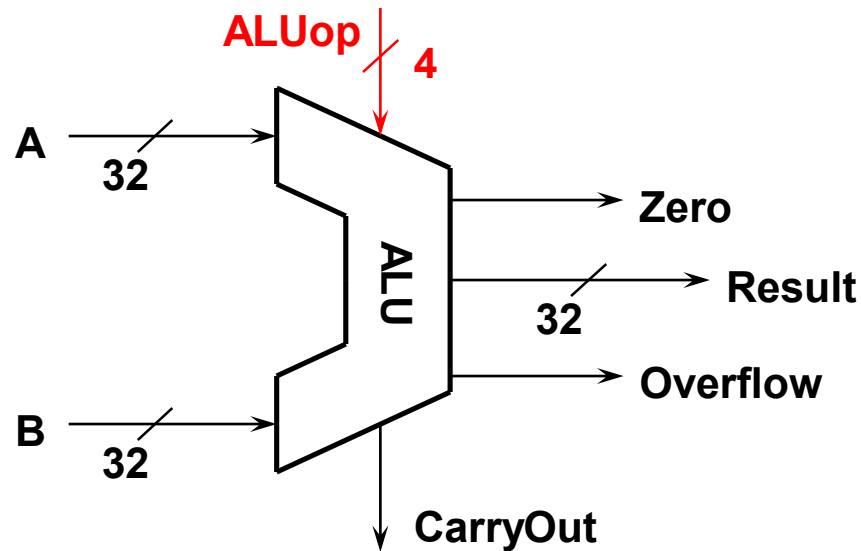
◆ Bit 0 in ALU



A Ripple Carry Adder and Set on Less Than



Functional Specification



ALU Control (ALUop)

0000

0001

0010

0110

0111

1100

Function

and

or

add

subtract

set-on-less-than

nor

Overflow

有號數相加、減，才有 Overflow 的問題 (無號數相加，頂多超過範圍，多產生一個進位)

Decimal	Binary	Decimal	2's complement
0	0000	0	0000
1	0001	-1	1111
2	0010	-2	1110
3	0011	-3	1101
4	0100	-4	1100
5	0101	-5	1011
6	0110	-6	1010
7	0111	-7	1001
		-8	1000

Overflow:

正+正

負+負

正-負

負-正

→ 這4 Case
可能產生 Overflow

* Review
Get 2's Comp

01110

找到第一個1



100010

取反

不動

Ex: 7 + 3 = 10 but ...

	0	1	1	1	1	7 正
+	0	0	0	1	1	3 正
	1	0	1	0		-6

↓ 1st Way: 判斷 Sign Bit

- 4 - 5 = - 9 but ...

	1	0	0	0	0	-4 負
+	1	1	0	1	1	-5 負
	0	1	1	1		7

2st: 判斷進位 (最後進位 & 最後1 Bit 的進位是否相等)

Overflow Detection

◆ Overflow: result too big/small to represent

- $-8 \leq 4\text{-bit binary number} \leq 7$
- When adding operands with different signs, overflow cannot occur!

• Overflow occurs when adding:

- 2 positive numbers and the sum is negative
- 2 negative numbers and the sum is positive

=> sign bit is set with the value of the result

→ Sign Bit 變值的一部分

- Overflow if: Carry into MSB $\neq$ Carry out of MSB (用進位判斷)

	0	1	1	1	1	7
	0	1	1	1		
+	0	0	1	1		3
	1	0	1	0		-6

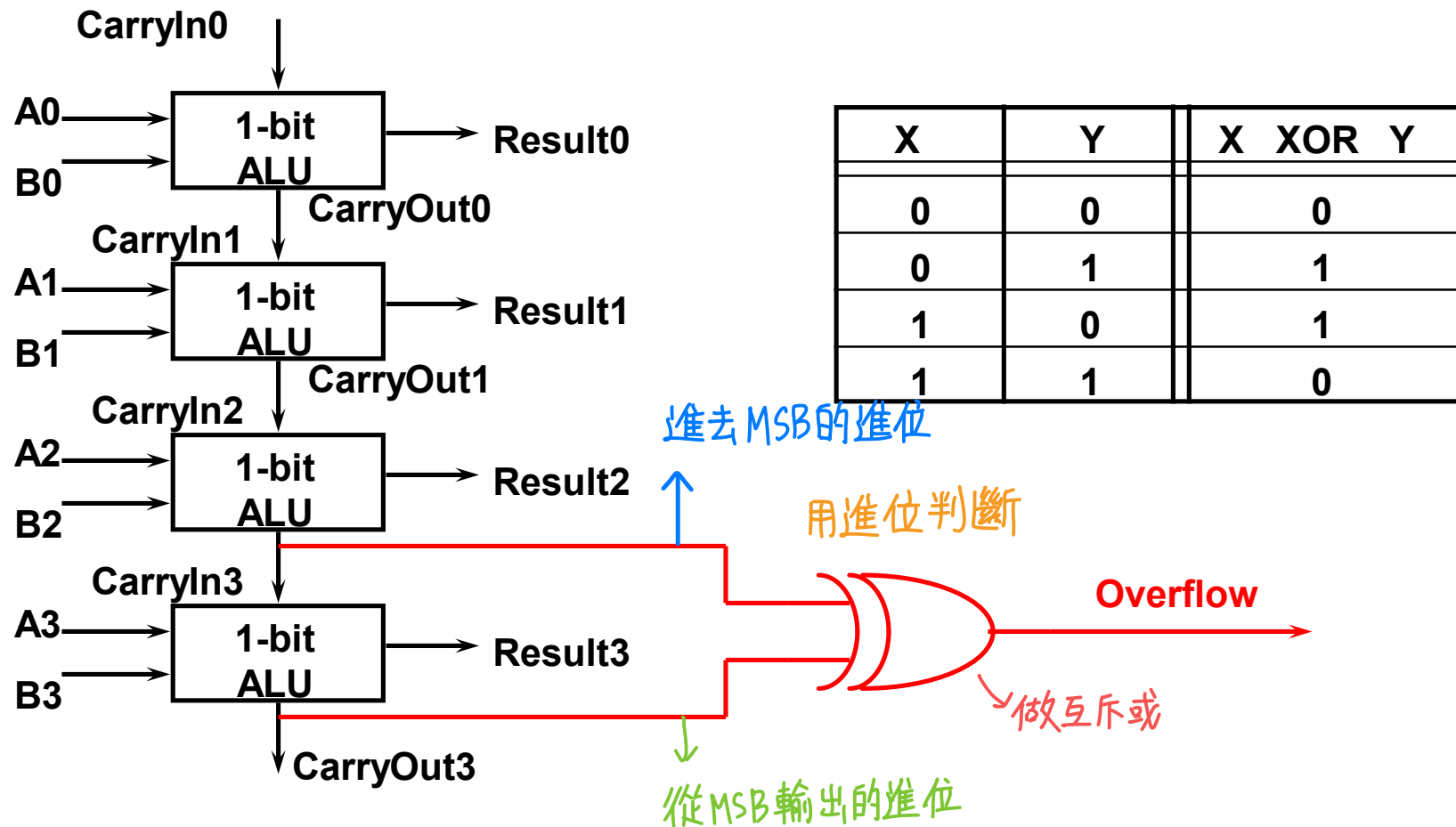
2補數: -6

照常算2進位(無號數): 10

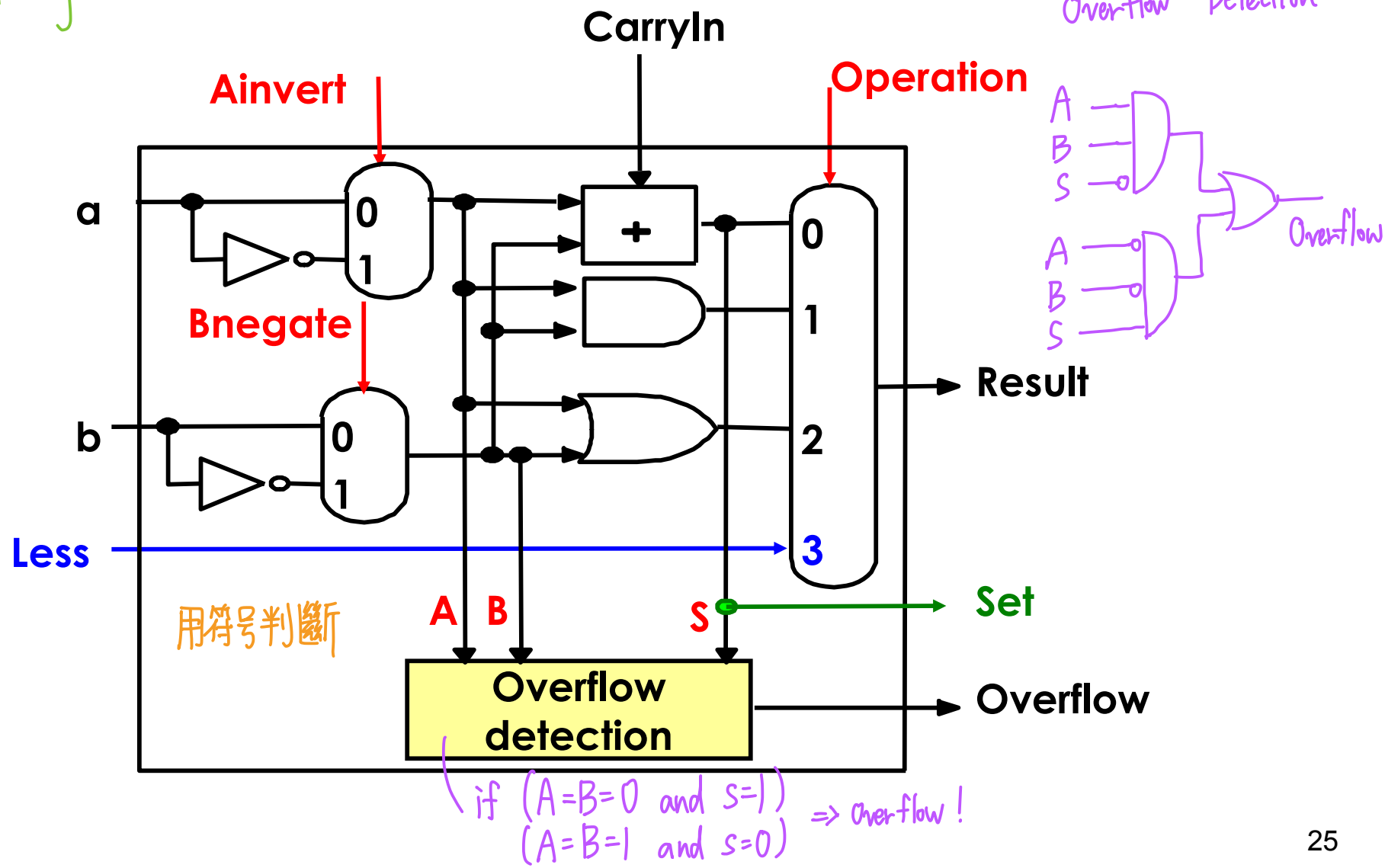
	1	0	0	0		-4
	1	1	0	0		
+	1	0	1	1		-5
	0	1	1	1		7

Overflow Detection Logic

- ◆ **Overflow = CarryIn[N-1] XOR CarryOut[N-1]**



* Sign Bit ALU

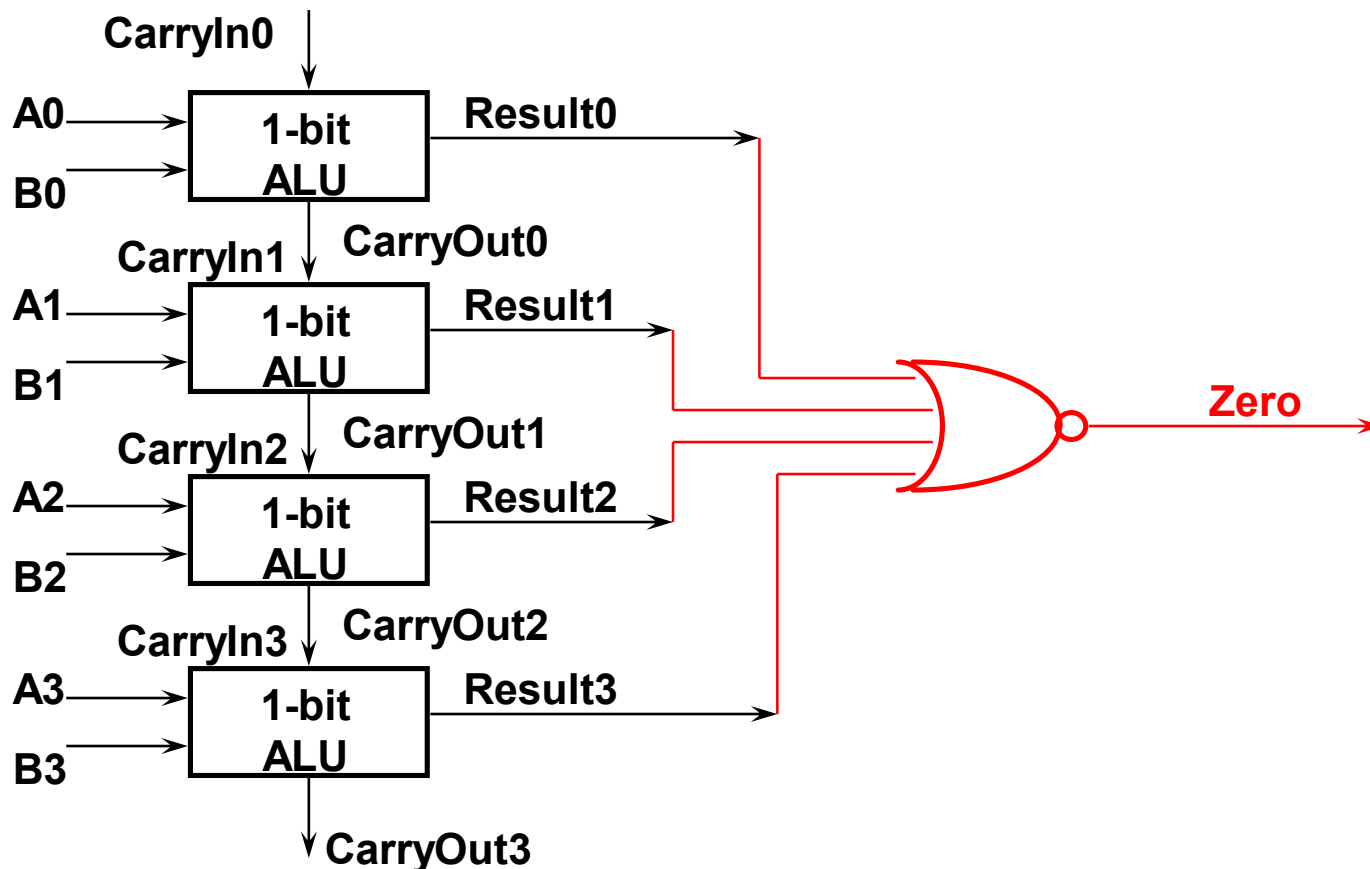


Dealing with Overflow

- ◆ Some languages (e.g., C) ignore overflow
- ◆ Other languages (e.g., Ada, Fortran) require raising an exception(例外處理)

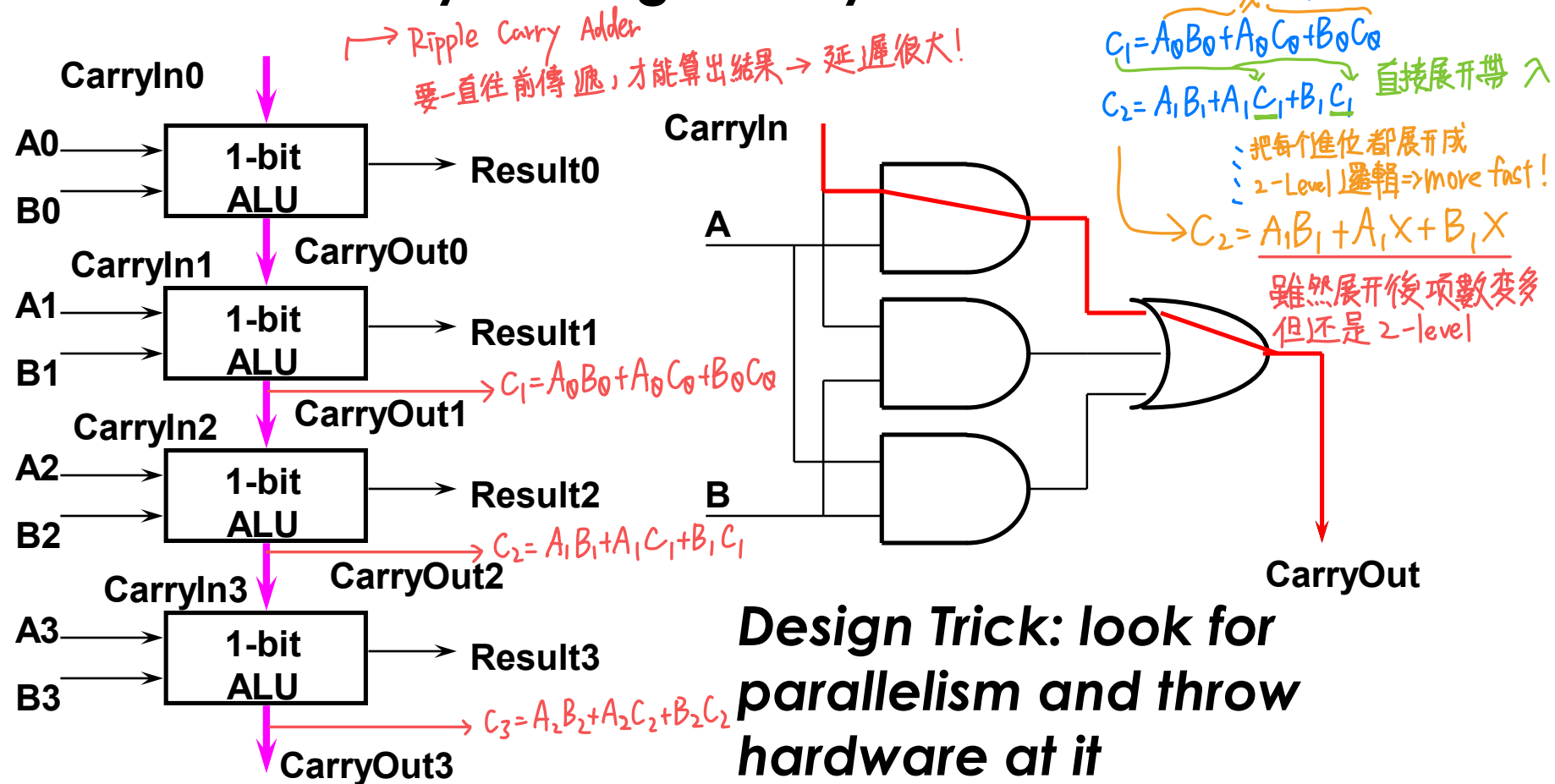
Zero Detection Logic

- ◆ Zero Detection Logic is a one BIG NOR gate (support conditional jump)



Problems with Ripple Carry Adder

- ◆ Carry bit may have to propagate from LSB to MSB => worst case delay: N-stage delay

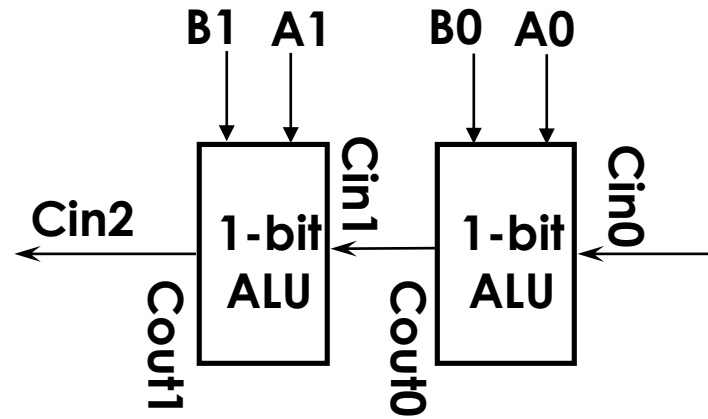


Outline

- ◆ Addition and subtraction
- ◆ Constructing an arithmetic logic unit
 - Building ALU
 - Add, sub, and, or, nor
 - Set-on-less-than, overflow detection, zero detection
 - Fast adders
 - Cascaded carry look-ahead adder 可串接式前瞻進位加法器
- ◆ Multiplication
 - Unsigned multiply
 - Signed multiply
- ◆ Division
- ◆ Floating point
 - Representations
 - Addition and multiplication

Carry Look-ahead: Theory (I)

(Appendix C)



◆ $\text{CarryOut} = (B * \text{CarryIn}) + (A * \text{CarryIn}) + (A * B)$

- $\text{Cin1} = \text{Cout0} = (B0 * \text{Cin0}) + (A0 * \text{Cin0}) + (A0 * B0)$
- $\text{Cin2} = \text{Cout1} = (B1 * \text{Cin1}) + (A1 * \text{Cin1}) + (A1 * B1)$

◆ Substituting Cin1 into Cin2:

- $$\begin{aligned} \text{Cin2} = & (A1 * A0 * B0) + (A1 * A0 * \text{Cin0}) + (A1 * B0 * \text{Cin0}) \\ & + (B1 * A0 * B0) + (B1 * A0 * \text{Cin0}) + (B1 * B0 * \text{Cin0}) \\ & + (A1 * B1) \end{aligned}$$

2-level
Delay

Cin 1
疊代進 Cin2

Carry Look-ahead: Theory (II)

◆ Now define two new terms:

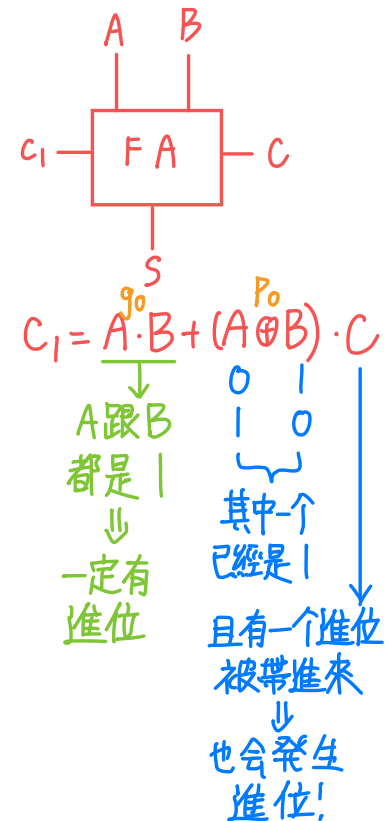
- ^{產生的進位} Generate Carry at Bit i: $g_i = A_i * B_i$ (AND)
- ^{準備產生的進位} Propagate Carry via Bit i: $p_i = A_i \text{ xor } B_i$

◆ We can rewrite:

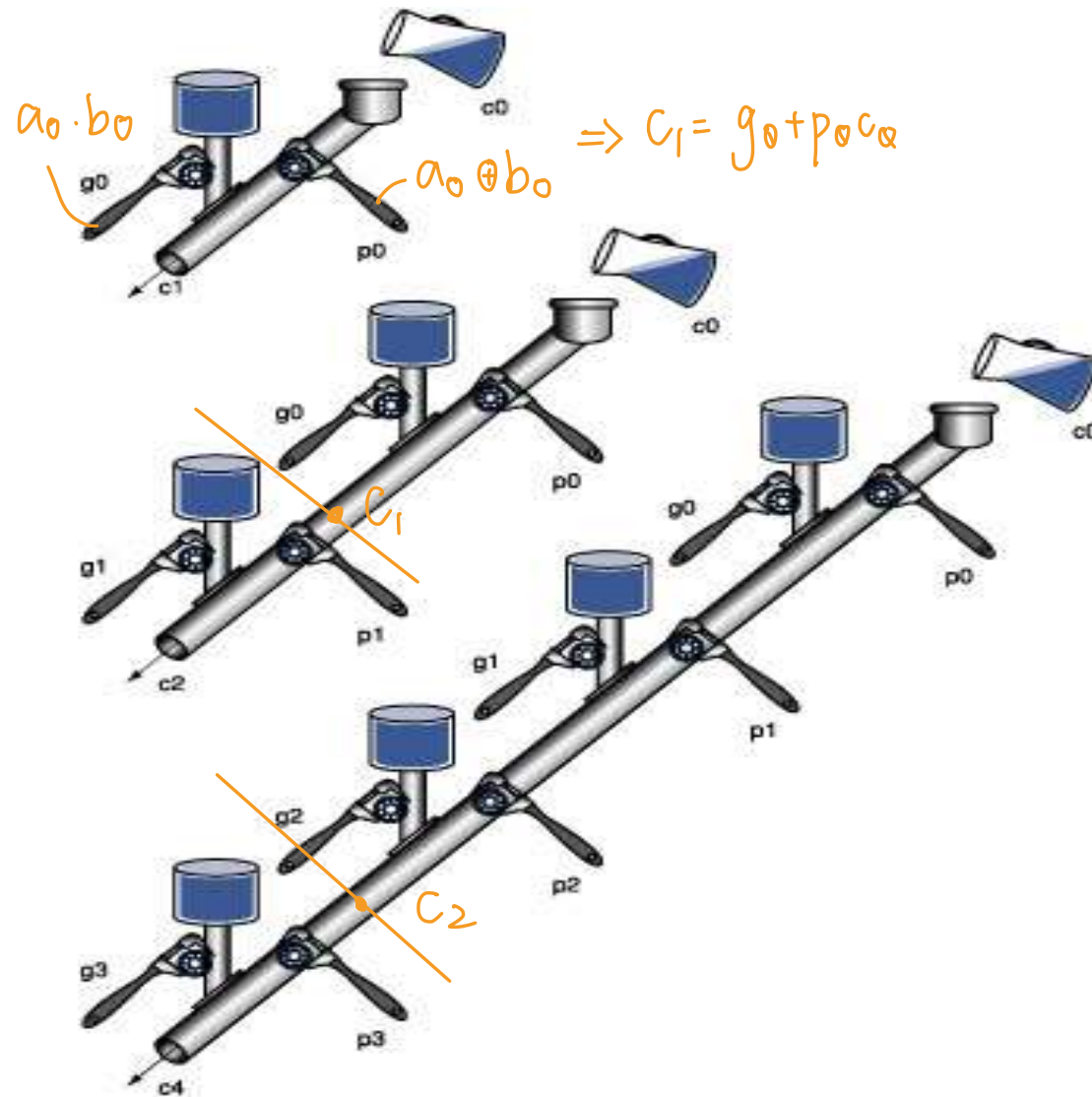
- $C_{in1} = g_0 + (p_0 * C_{in0})$
- $C_{in2} = g_1 + (p_1 * g_0) + (p_1 * p_0 * C_{in0})$ ($C_{in2} = g_1 + (p_1 * C_{in1}) \Rightarrow$ 展开)
- $C_{in3} = g_2 + (p_2 * g_1) + (p_2 * p_1 * g_0) + (p_2 * p_1 * p_0 * C_{in0})$

◆ Carry going into bit 3 is 1 if

- We generate a carry at bit 2 (g_2)
- Or we generate a carry at bit 1 (g_1) and bit 2 allows it to propagate ($p_2 * g_1$)
- Or we generate a carry at bit 0 (g_0) and bit 1 as well as bit 2 allows it to propagate



A Plumbing Analogy for Carry Look-ahead (1, 2, 4 bits)

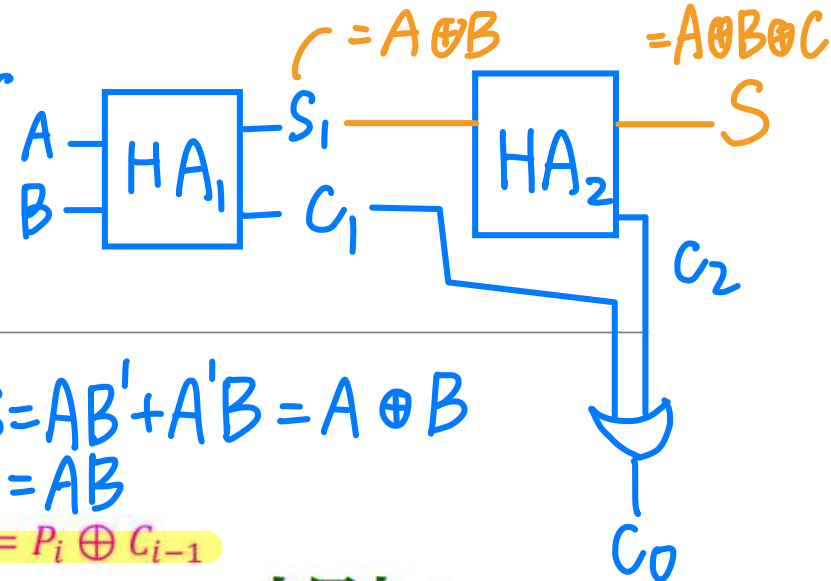


Carry Look-ahead Adder

- ◆ Expensive to build a “full” carry look-ahead adder
 - Ex: $Cin3 = g2 + (p2 * g1) + (p2 * p1 * g0) + (p2 * p1 * p0 * Cin0)$
 - Just imagine length of the equation for Cin31

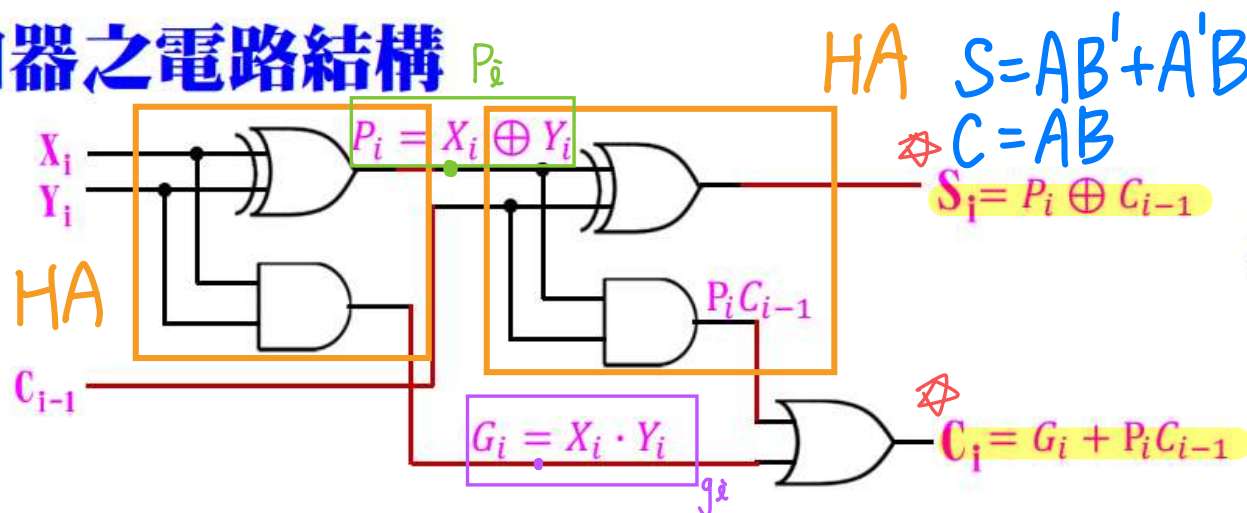
会有32项

Full Adder



前瞻式加法器設計原理

全加器之電路結構

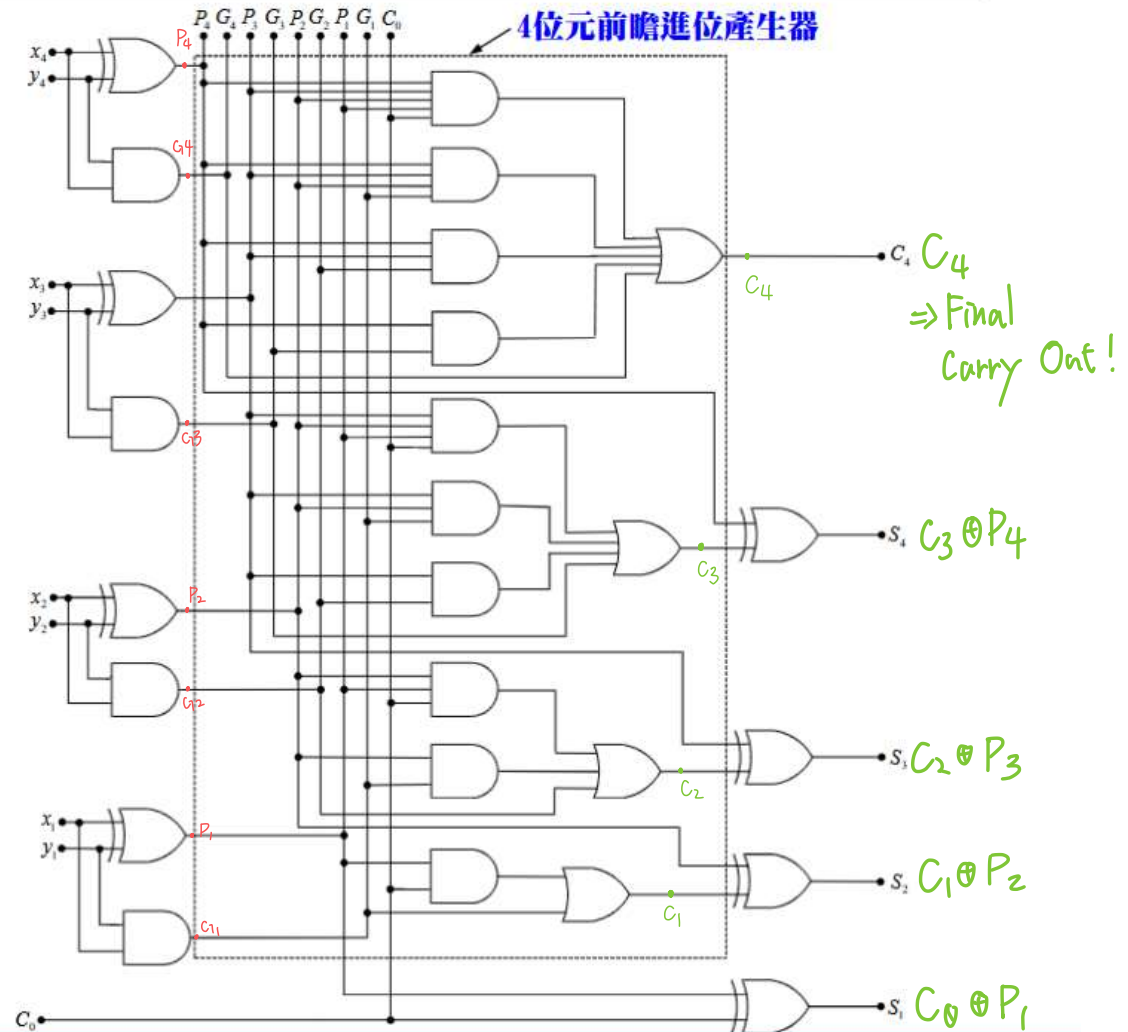
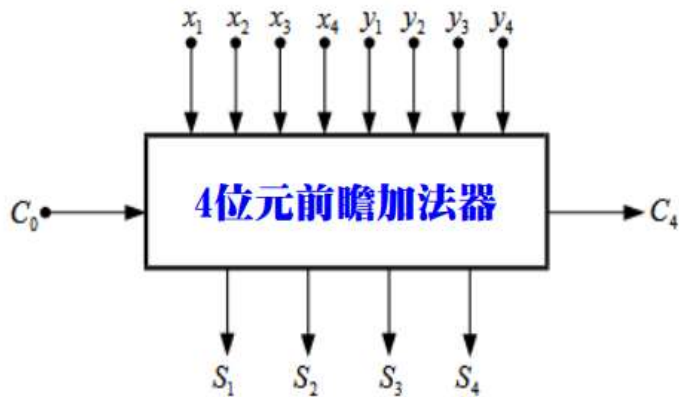


左圖中：
i 代表目前位元
i - 1 代表前一位元。

以4位元加法器為例，由上圖可得C1、C2、C3、C4分為別：

各級進位輸出結果	注意到！全部展开就是P&G的組合 (邏輯gate只有AND、XOR)	各級之和輸出
$C_1 = G_1 + P_1 C_0$		$S_1 = P_1 \oplus C_0$
$C_2 = G_2 + P_2 C_1 = G_2 + P_2 (G_1 + P_1 C_0) \Rightarrow$ 乘開後共3項		$S_2 = P_2 \oplus C_1$
$C_3 = G_3 + P_3 C_2 = G_3 + P_3 (G_2 + P_2 (G_1 + P_1 C_0)) \Rightarrow$ 乘開後共4項		$S_3 = P_3 \oplus C_2$
$C_4 = G_4 + P_4 C_3 = G_4 + P_4 (G_3 + P_3 (G_2 + P_2 (G_1 + P_1 C_0))) \Rightarrow$ 乘開後5項		$S_4 = P_4 \oplus C_3$

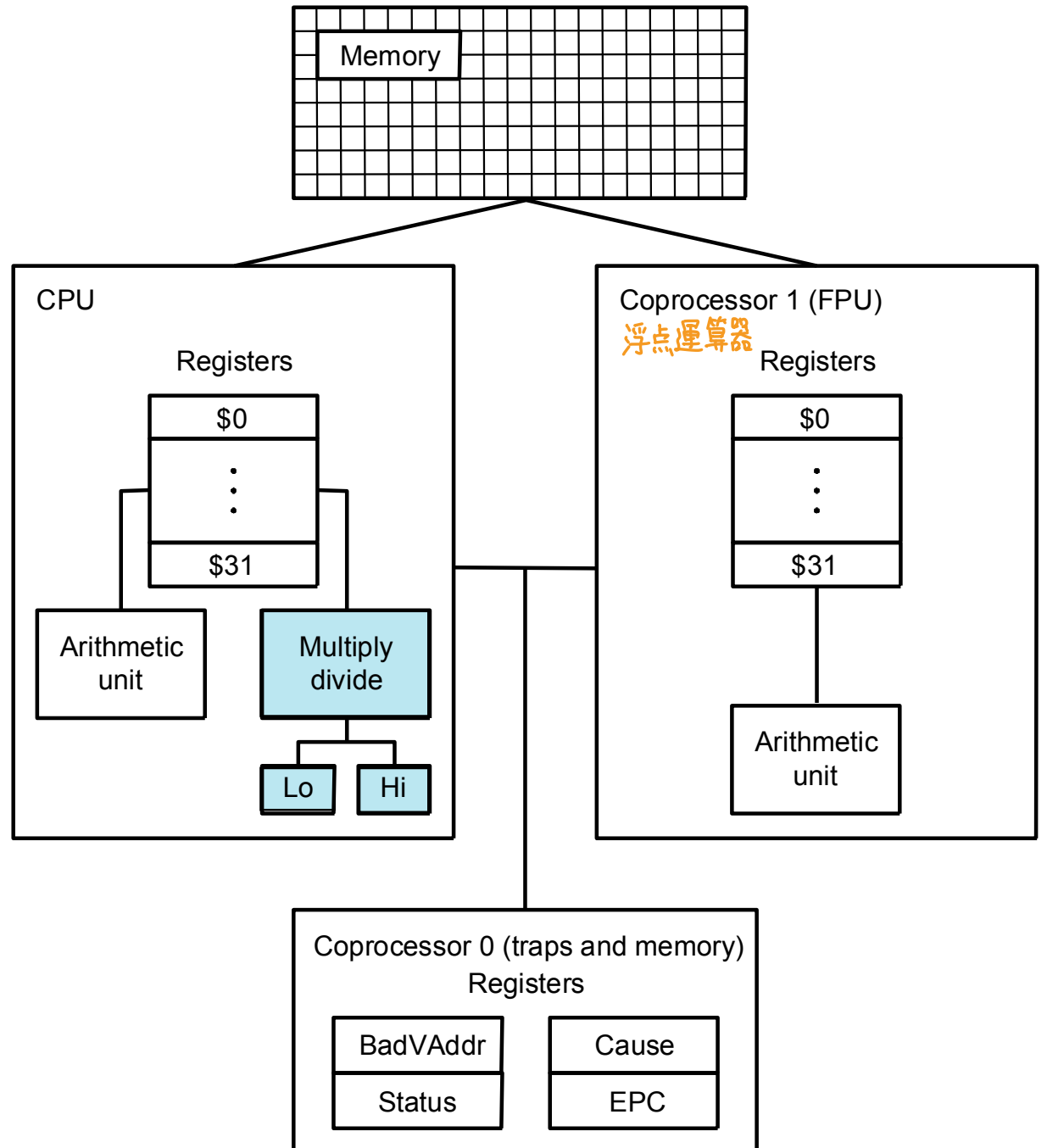
前瞻式加法器電路方塊圖及內部結構



Outline

- ◆ Addition and subtraction
- ◆ Constructing an arithmetic logic unit
 - Building ALU
 - Add, sub, and, or, nor
 - Set-on-less-than, overflow detection, zero detection
 - Fast adders
 - Cascaded carry look-ahead adder
 - Multiple level carry look-ahead adder
- ◆ Multiplication
 - Unsigned multiply
 - Signed multiply
- ◆ Division
- ◆ Floating point
 - Representations
 - Addition and multiplication

MIPS R2000 Organization



Multiplication in MIPS

`mult $t1, $t2` `# t1 * t2`

- ◆ No destination register: product could be $\sim 2^{64}$; need two special registers to hold it
- ◆ 3-step process:

Hi Lo Register 儲存

`$t1` 01111111111111111111111111111111

X `$t2` 01000000000000000000000000000000

00011111111111111111111111111111 | 11000000000000000000000000000000

Hi

Lo

`mfhi $t3` `$t3` 00011111111111111111111111111111

`mflo $t4` `$t4` 11000000000000000000000000000000

乘完還要
移出來

要先乘!

MIPS Multiplication

- ◆ Two 32-bit registers for product
 - HI: most-significant 32 bits
 - LO: least-significant 32-bits
- ◆ Instructions
 - mult rs, rt / multu rs, rt
 - 64-bit product in HI/LO $\Rightarrow$ Hi、Lo 組成一個64 Bit 的乘積項
 - mfhi rd / mflo rd
 - Move from HI/LO to rd
 - Can test HI value to see if product overflows 32 bits
 - mul rd, rs, rt $\rightarrow$ 他會把rsxrt相乘結果的較低的32位元(Lo)存入rd
 - Least-significant 32 bits of product $\rightarrow$ rd

Unsigned Multiply

◆ Paper and pencil example (unsigned):

被乘數 **Multiplicand**
乘數 **Multiplier**

$$\begin{array}{r} 1000_{\text{ten}} \\ \times 1001_{\text{ten}} \\ \hline 1000 \\ 0000 \\ 0000 \\ 1000 \\ \hline 01001000_{\text{ten}} \end{array}$$

→ 要先判斷乘數的每個Bit是1 or 0?

判斷中間的Bit很困難, 但判斷右邊的Bit很簡單! ⇒ shift Register!

⇒ BIN乘法沒有真的「乘」而只有加法 & 移位!

Product

◆ m bits x n bits = m+n bit product

◆ Binary makes it easy:

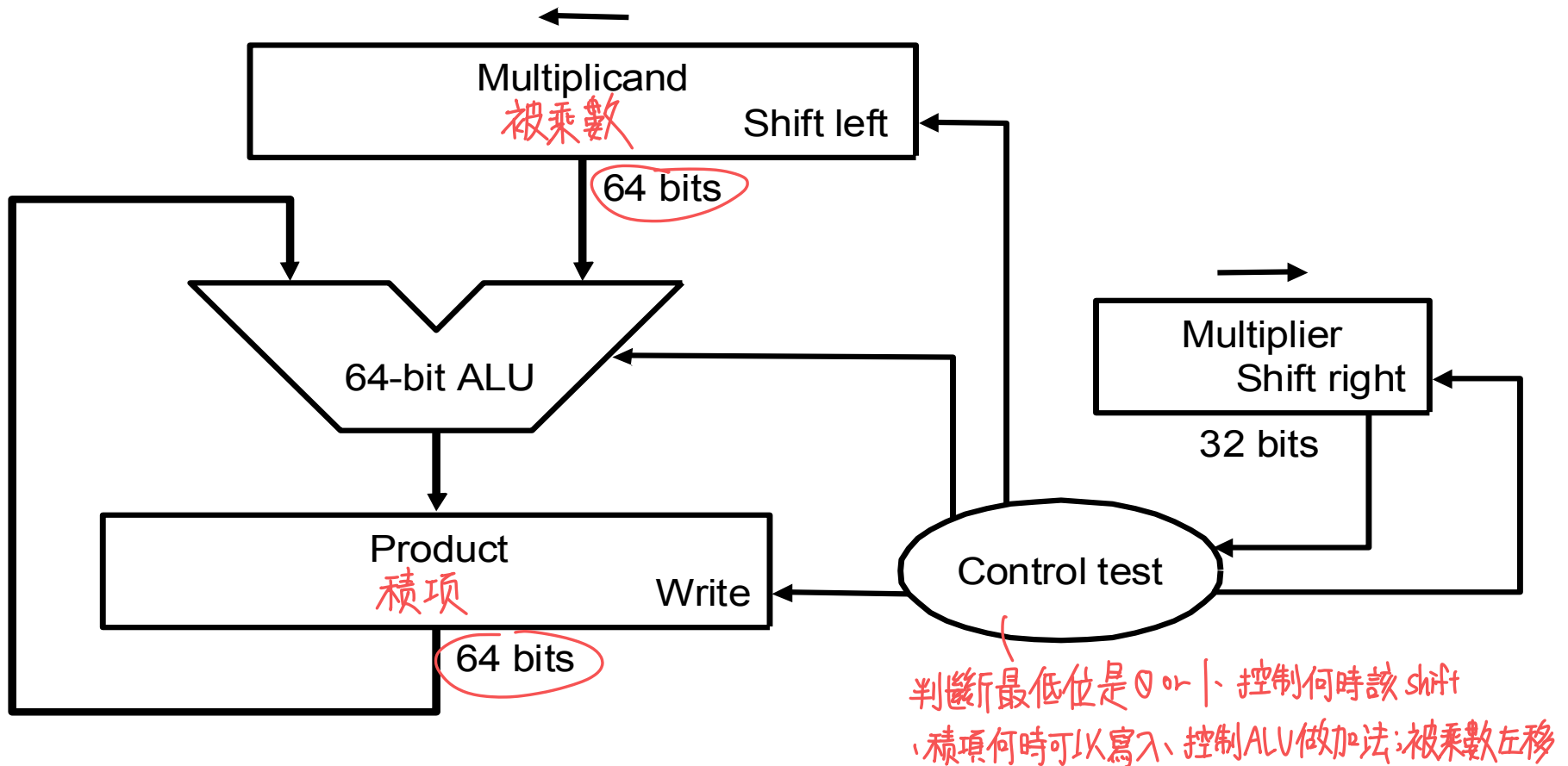
- 0 ⇒ place 0 (0 x multiplicand) ⇒ 看到0, 全變0放下來!

- 1 ⇒ place a copy (1 x multiplicand) ⇒ 看到1, copy一份放下來

◆ 2 versions of multiply hardware and algorithm

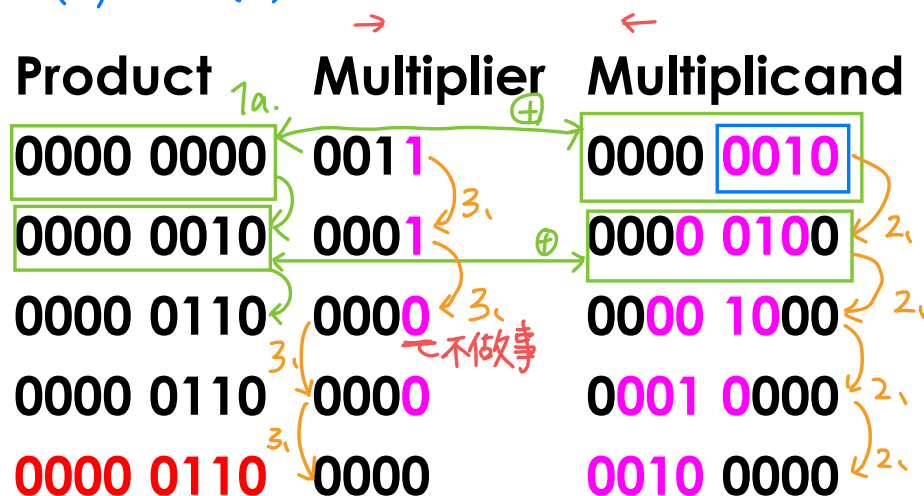
Unsigned Multiplier (Ver. 1)

- ◆ 64-bit multiplicand register (with 32-bit multiplicand at right half), 64-bit ALU, 64-bit product register, 32-bit multiplier register



很重要

被乗數 乘數
0010 x 0011
(2) (3)

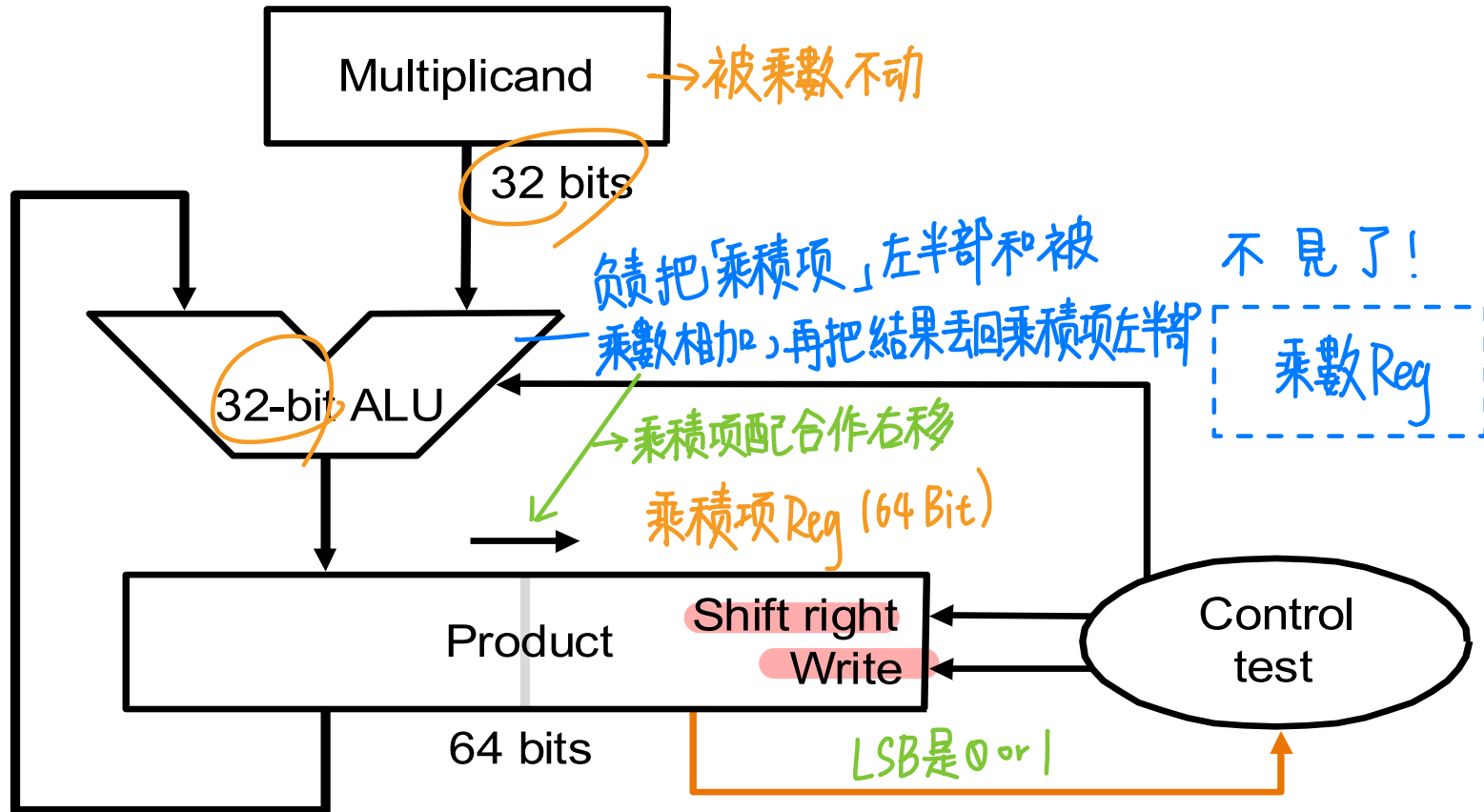


Observations: Multiply Ver. 1

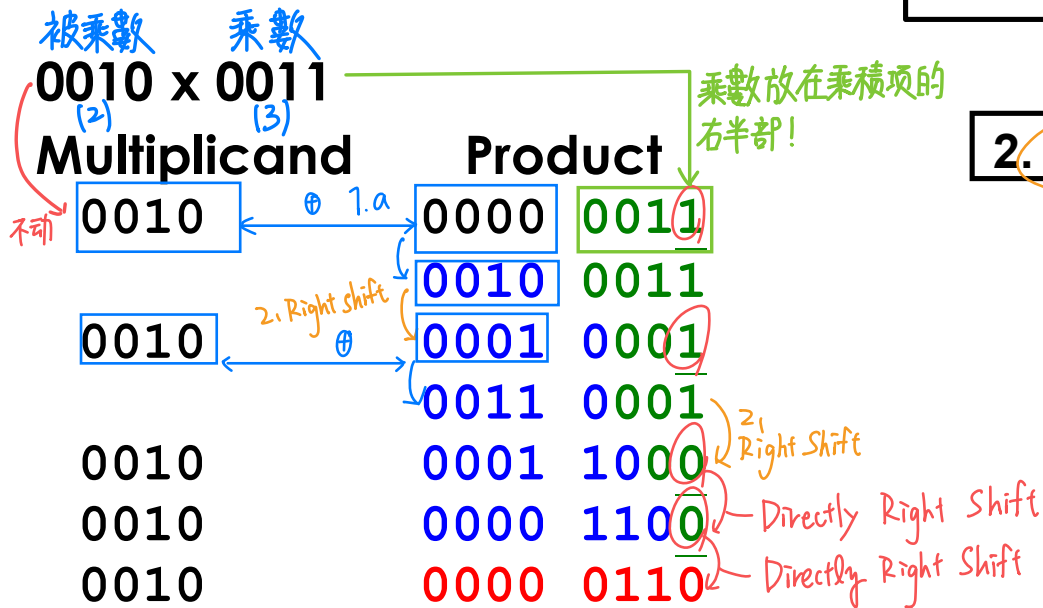
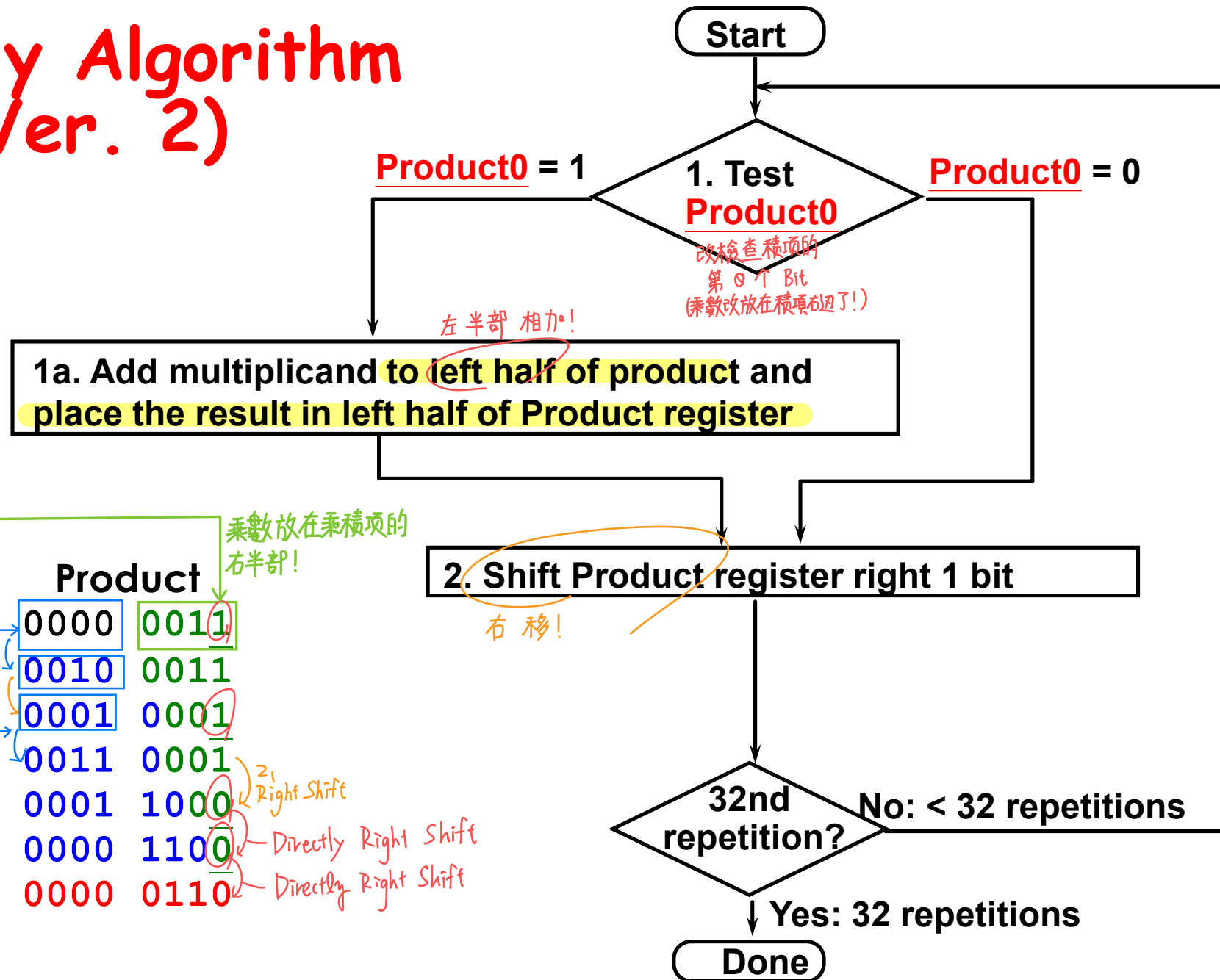
- ◆ Delay ratio of multiply to add 5:1 to 100:1 乘法 & 加法約是5:1 ~ 100:1 的速度差
- ◆ Half of the bits in multiplicand always 0 → ver1 的 Algo 有一半都是0 ⇒ 非常浪費
⇒ 64-bit adder is wasted
- ◆ 0's inserted in right of multiplicand as shifted
⇒ least significant bits of product never changed once formed → 乘積項的LSB都不會改變!
- ◆ Instead of shifting multiplicand to left, shift product to right? ⇒ 把「被乘數的左移」取代為「乘積項右移」
- ◆ Product register wastes space ⇒ combine Multiplier and Product register
⇒ 「乘積Reg」非常浪費空間! ⇒ 把「乘數」&「乘積項」擺在一起

Unsigned Multiplier (Ver. 2)

- ◆ 32-bit Multiplicand register, 32-bit ALU, 64-bit Product register (HI & LO in MIPS), (0-bit Multiplier register)



Multiply Algorithm (Ver. 2)



Outline

- ◆ Addition and subtraction
- ◆ Constructing an arithmetic logic unit
 - Building ALU
 - Add, sub, and, or, nor
 - Set-on-less-than, overflow detection, zero detection
 - Fast adders
 - Cascaded carry look-ahead adder
 - Multiple level carry look-ahead adder
- ◆ Multiplication
 - Unsigned multiply
 - Signed multiply
- ◆ Division
- ◆ Floating point
 - Representations
 - Addition and multiplication

Signed Multiply

有號數乘法

◆ What about signed multiplication?

先變兩個
正數相乘

Way 1 • The easiest solution is to make both positive and remember whether to complement product when done (leave out sign bit, run for 31 steps)

最後來看要不要把
乘積項取補

Way 2 • Apply definition of 2's complement

■ sign-extend partial products and subtract at end

(符號擴充部分積, 並在最後減去)

用二補數的定義做

Using Definition of 2's Complement

◆ Paper and pencil example (signed):

Multiplicand

Multiplier

$$\begin{array}{r}
 1001 \text{ (-7)} \\
 \times 1001 \text{ (-7)} \\
 \hline
 11111001 \quad \text{前面幾項相力} \\
 + 00000000 \\
 + 00000000 \\
 + 11001 \quad \text{最後一項用減的} \rightarrow - \\
 \hline
 00110001 \text{ (49)}
 \end{array}$$

Handwritten notes:
 - 前面幾項相力 (Previous terms cancel out)
 - 最後一項用減的 (Last term is subtracted)
 - 把1001下放, 做 Sign-Extend (Bring down 1001, do Sign-Extend)
 - 要同時 Left Shift & Sign Extend (Need to do Left Shift & Sign Extend simultaneously)

$$\begin{array}{r}
 * +5: 0101 \\
 2's \rightarrow 1011
 \end{array}$$

$$\begin{array}{r}
 +6 \times -5 \\
 0110 \quad 1011
 \end{array}$$

$$\begin{array}{r}
 0110 \\
 \times 1011 \\
 \hline
 \end{array}$$

◆ Rule 1: **Multiplicand sign extended**

◆ Rule 2: **Sign bit (s) of Multiplier**

● 0 => 0 x multiplicand

● 1 => -1 x multiplicand

$$\begin{array}{r}
 00000110 \\
 + 0000110 \\
 + 0000000 \\
 - 00110 \\
 \hline
 00010010 \\
 - 00110000 \\
 \hline
 -30 \leftarrow 11100010 \\
 30 (00011110)
 \end{array}$$

Booth's Algorithm: Motivation

- ◆ Example: $2 \times 6 = 0010 \times 0110$:

Booth's Algo Skip

	0010 _{two}	
x	0110 _{two}	
+	0000	shift (0 in multiplier)
+	0010	add (1 in multiplier)
+	0010	add (1 in multiplier)
+	0000	shift (0 in multiplier)
	0001100 _{two}	

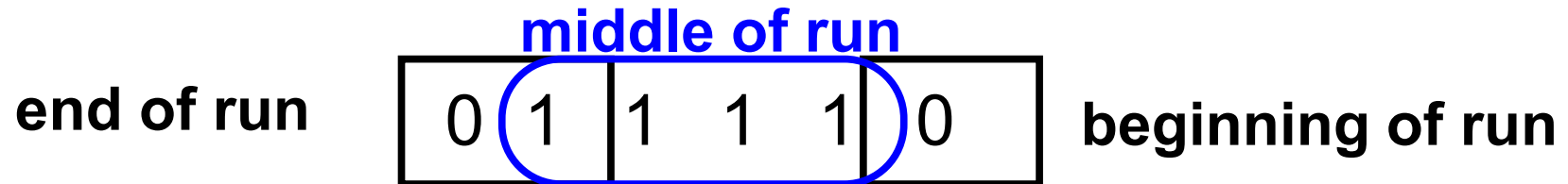
- ◆ Can get same result in more than one way:

$$6 = -2 + 8 \qquad 0110 = -00010 + 01000$$

- ◆ Basic idea: replace a string of 1s with an initial subtract on seeing a one and add after last one

	0010 _{two}	
x	0110 _{two}	
	0000	shift (0 in multiplier)
-	0010	sub (first 1 in multiplier)
	0000	shift (mid string of 1s)
+	0010	add (prior step had last 1)
	00001100 _{two}	

Booth's Algorithm: Rationale



Current Bit to bit	right	Explanation	Example	Op
1	0	Begins run of 1s	0000111 <u>1</u> 000	sub
1	1	Middle run of 1s	00001 <u>11</u> 1000	none
0	1	End of run of 1s	000 <u>01</u> 111000	add
0	0	Middle run of 0s	00 <u>00</u> 1111000	none

Originally for speed (when shift was faster than add)

♦ Why it works?

$$\begin{array}{r}
 -1 \\
 + 10000 \\
 \hline
 01111
 \end{array}$$

Booth's Algorithm

1. Depending on the current and previous bits, do one of the following:
 - 00: Middle of a string of 0s, no arithmetic op.
 - 01: End of a string of 1s, so add multiplicand to the left half of the product
 - 10: Beginning of a string of 1s, so subtract multiplicand from the left half of the product
 - 11: Middle of a string of 1s, so no arithmetic op.
2. As in the previous algorithm, shift the Product register right (arithmetically) 1 bit

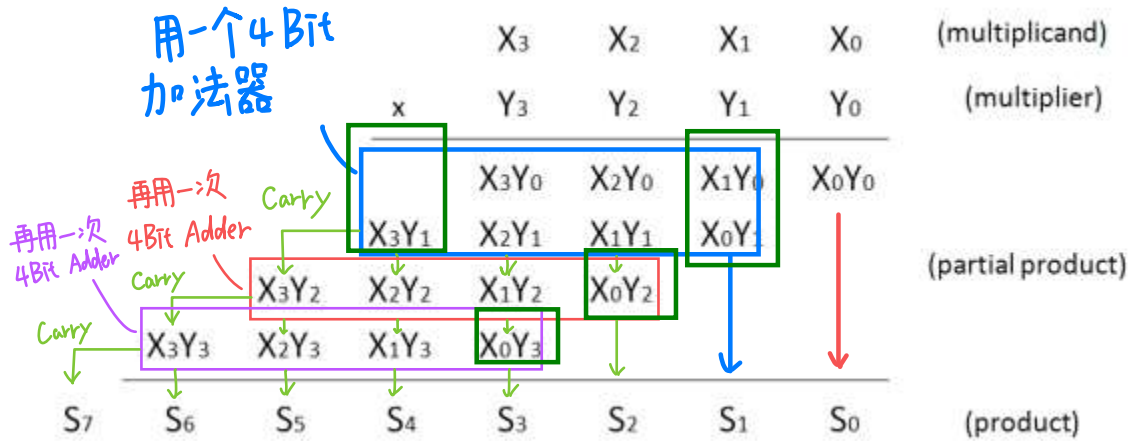
Booths Example (2 x 7)

Operation	Multiplicand	Product	next?
0. initial value	0010	0000 0111 0	10 -> sub
1a. $P = P - m$	1110	+1110	
		1110 0111 0	shift P (sign ext)
1b.	0010	<u>1</u> 111 0011 1	11 -> nop, shift
2.	0010	1111 1 001 1	11 -> nop, shift
3.	0010	1111 11 00 1	01 -> add
4a.	0010	+0010	
		0001 11 00 1	shift
4b.	0010	0000 111 0 0	done

Booths Example (2 x -3)

Operation	Multiplicand	Product	next?
0. initial value	0010	0000 1101 0	10 -> sub
1a. $P = P - m$	1110	+1110	
		1110 1101 0	shift P (sign ext)
1b.	0010	<u>1111</u> 0 110 1	01 -> add
	0010	+0010	
2a.		0001 0 110 1	shift P
2b.	0010	0000 10 11 0	10 -> sub
	1110	+1110	
3a.	0010	1110 10 11 0	shift
3b.	0010	1111 010 1 1	11 -> nop
4a.		1111 010 1 1	shift
4b.	0010	1111 1010 1	done

4bit 快速乘法器



→ 用3个4 Bit Adder 可以实现!

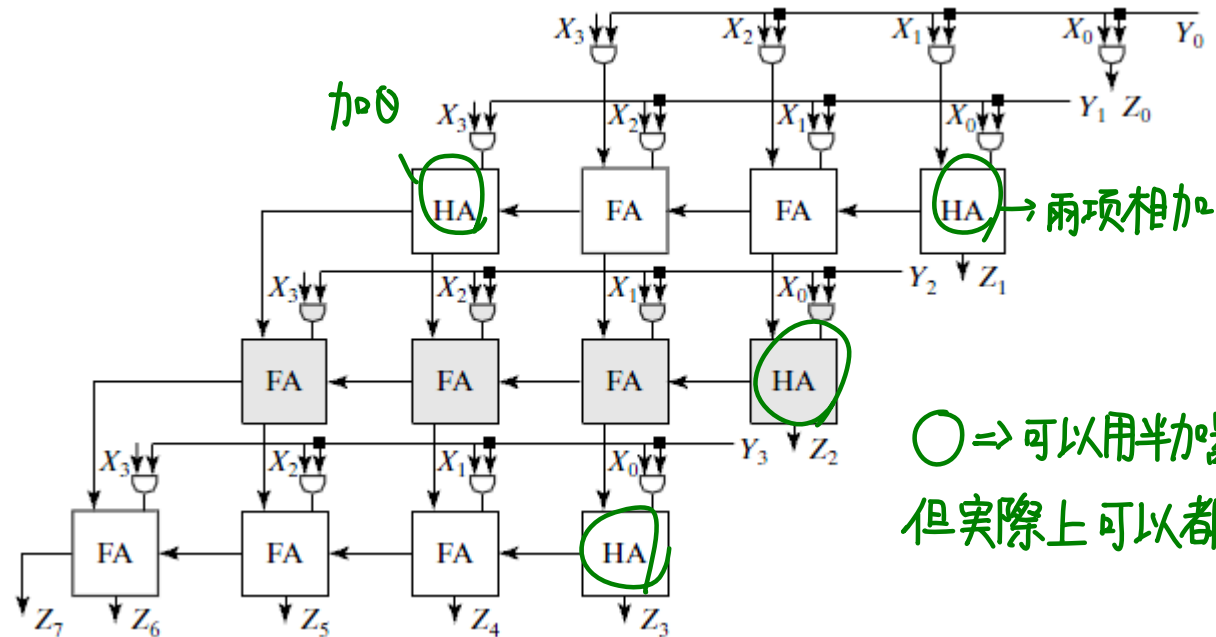
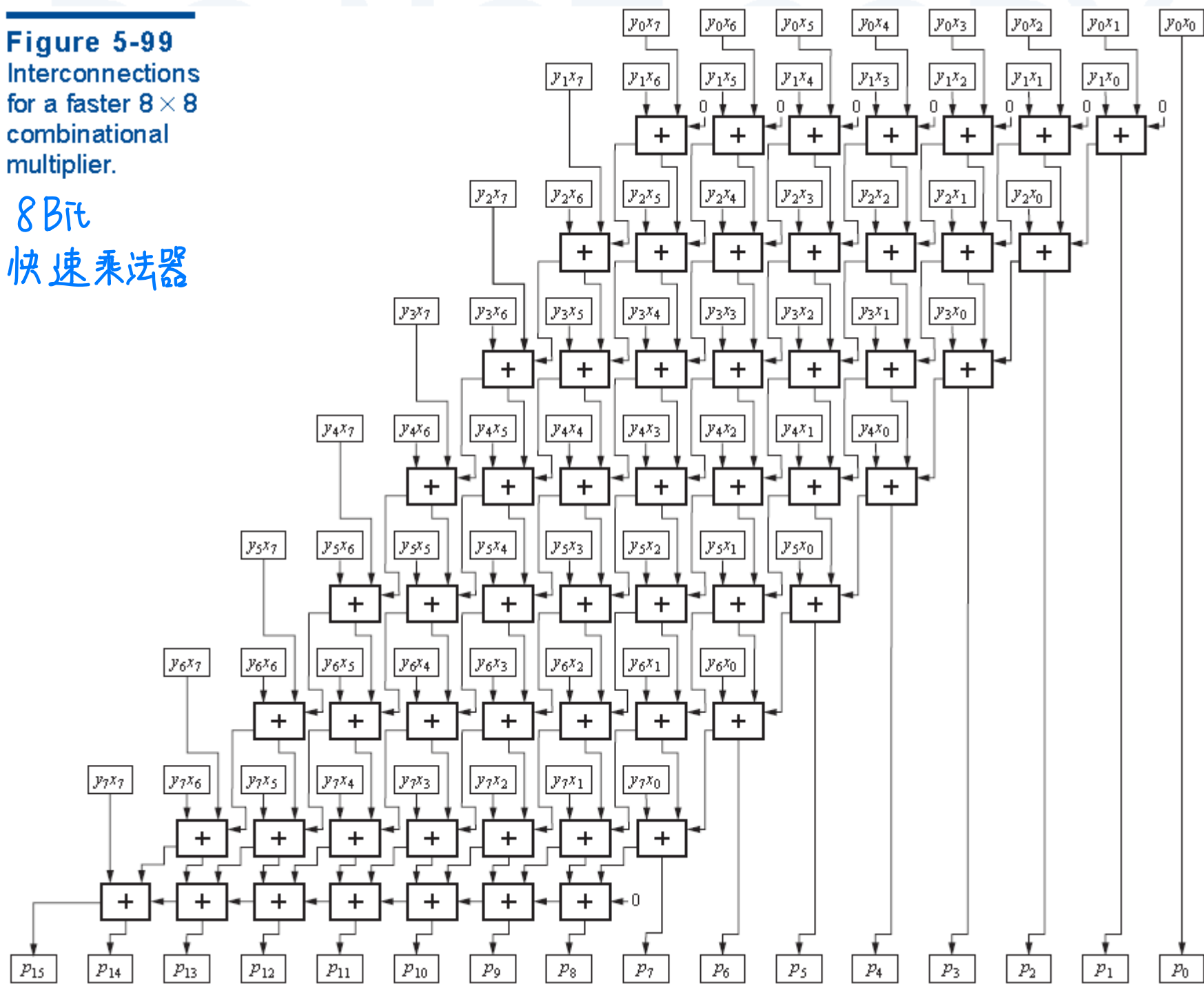


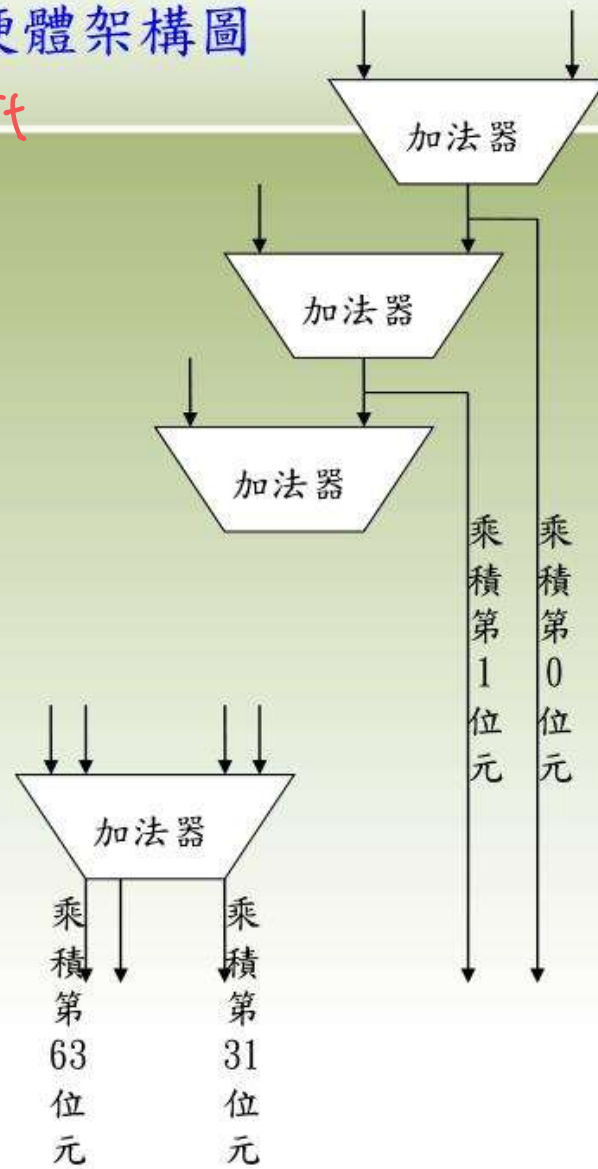
Figure 5-99
Interconnections
for a faster 8×8
combinational
multiplier.

8 Bit
快速乘法器



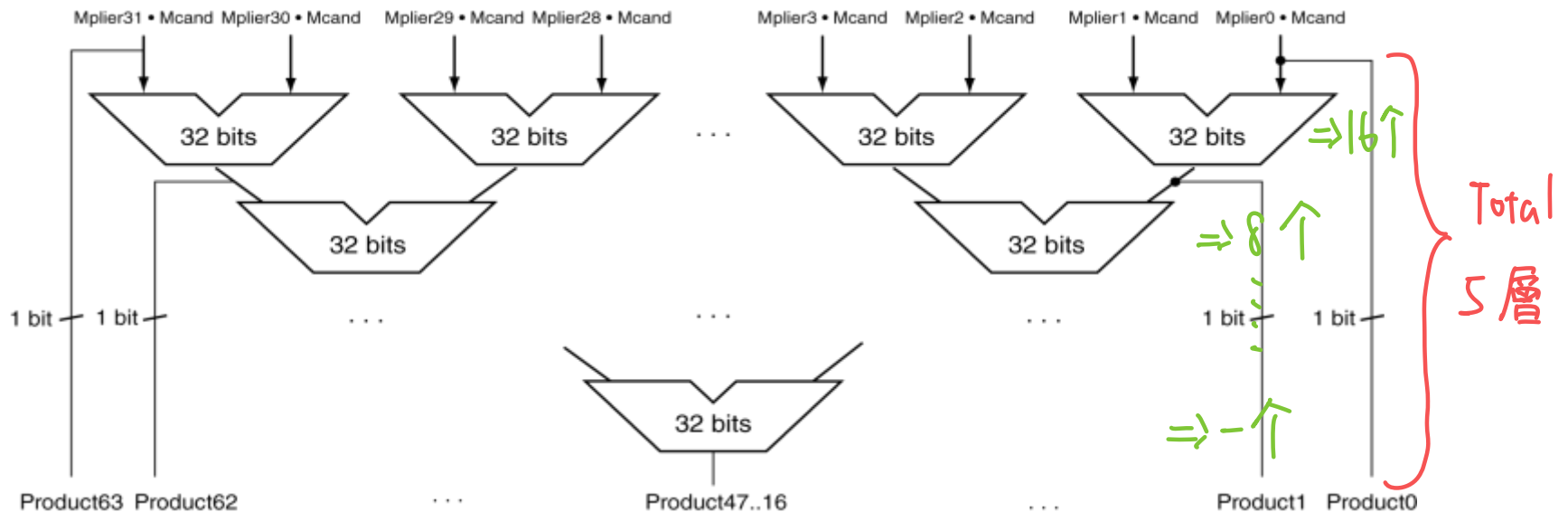
快速乘法器的硬體架構圖

32 bit x 32 bit



Faster Multiplier

- ◆ A combinational multiplier 平行 $\Rightarrow$ 拆成兩兩相加
- ◆ Use multiple adders
 - Cost/performance tradeoff

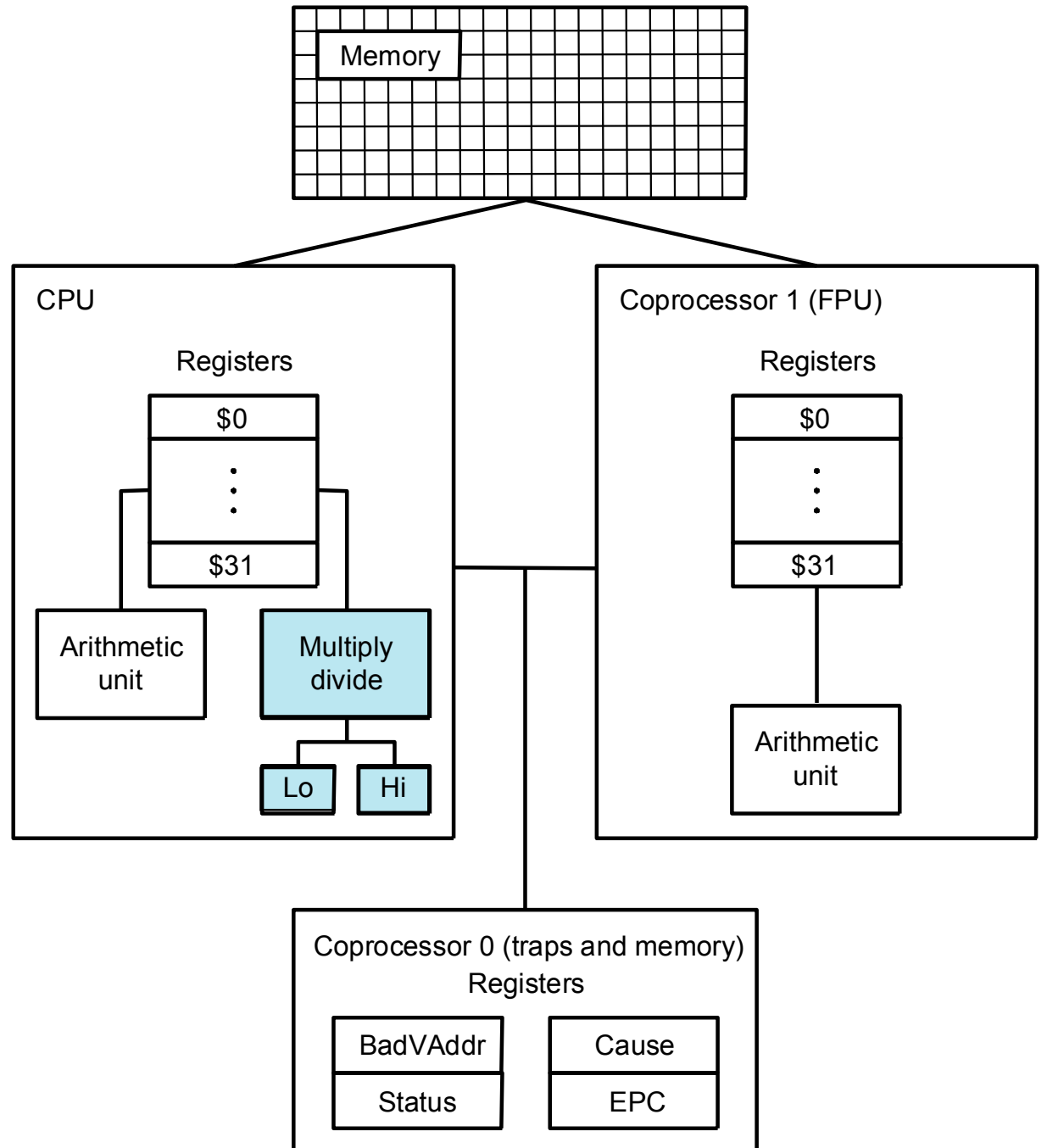


- ◆ Can be pipelined
 - Several multiplication performed in parallel

Outline

- ◆ Addition and subtraction (Sec. 3.2)
- ◆ Constructing an arithmetic logic unit (Appendix C)
 - Building ALU
 - Add, sub, and, or, nor
 - Set-on-less-than, overflow detection, zero detection
 - Fast adders
 - Cascaded carry look-ahead adder
 - Multiple level carry look-ahead adder
- ◆ Multiplication (Sec. 3.3, Appendix C)
 - Unsigned multiply
 - Signed multiply
- ◆ Division (Sec. 3.4)
- ◆ Floating point (Sec. 3.5)
 - Representations
 - Addition and multiplication

MIPS R2000 Organization



Division in MIPS

^{被除數} ^{除數}
`div $t1, $t2` # `t1 / t2`

- ◆ Quotient stored in Lo, remainder in Hi

`mflo $t3` #copy quotient to t3

`mfhi $t4` #copy remainder to t4

- ◆ 3-step process

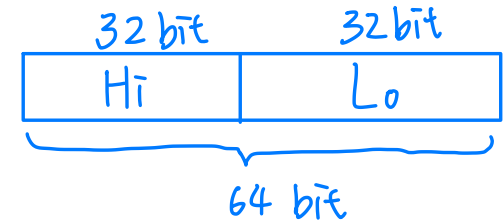
- ◆ Unsigned division:

`divu $t1, $t2` # `t1 / t2`

- Just like `div`, except now interpret `t1`, `t2` as unsigned integers instead of signed
- Answers are also unsigned, use `mfhi`, `mflo` to access

- ◆ No overflow or divide-by-0 checking (不檢查 overflow or 除0 交給 software 負責)

- Software must perform checks if required



Divide: Paper & Pencil

	$\begin{array}{r} 0001001_{\text{ten}} \\ \rightarrow \\ \hline 1001010_{\text{ten}} \\ -1000 \\ \hline 0010 \\ 0101 \\ 1010 \\ -1000 \\ \hline 10_{\text{ten}} \end{array}$	Quotient Dividend 把除數 不斷左移 比大小！
Divisor 1000 _{ten}		Remainder

- ◆ See how big a number can be subtracted, creating quotient bit on each step ↘ 一直找一個數字，減去除數，決定商數是0 or 1

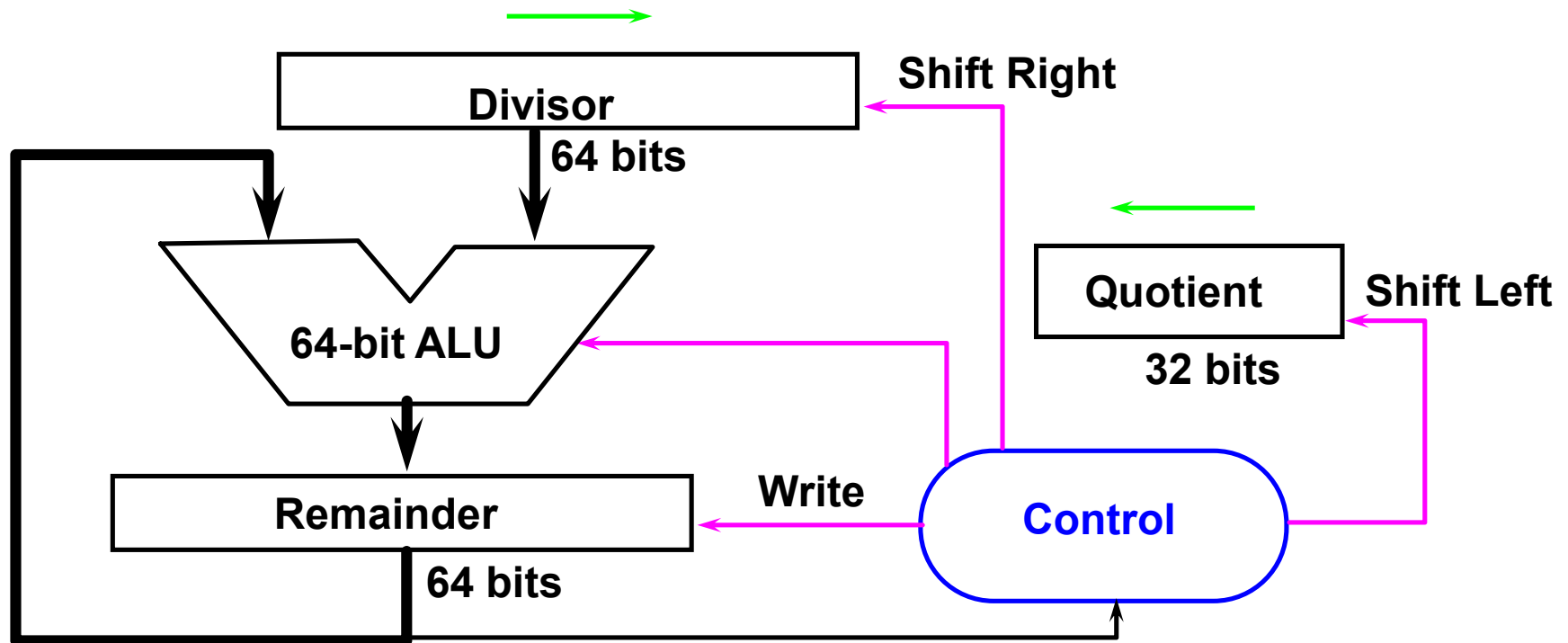
Binary => 1 * divisor or 0 * divisor

- ◆ Two versions of divide, successive refinement
- ◆ Both dividend and divisor are 32-bit positive integers

Divide Hardware (Version 1)

- ◆ 64-bit **Divisor register** (initialized with 32-bit divisor in left half), 64-bit ALU, 64-bit **Remainder register** (initialized with 64-bit dividend), 32-bit **Quotient register**

也许被除数是64 bit!



Divide Algorithm (Version 1)

0111 / 0010 4 bit → 5次

Quot.	Divisor	Rem.
0000	00100000	00000111

< 0, 負數! Restore

0000	00010000	00000111
------	----------	----------

左移一个 bit 右移 1 bit

0000	00001000	00000111
------	----------	----------

LSB 填 0 2次

0000	00000100	00000111
------	----------	----------

正數, 左移 1 bit 3次 LSB 填 1

0001	00000010	00000011
------	----------	----------

填 1

0011	00000001	00000001
------	----------	----------

5次

商: 3 餘: 1 → 7 ÷ 2 = 3 ... 1

Rem-Div

Rem-Div > 0! 33rd repetition?

Yes: 33 repetitions

Done

Start: Place Dividend in Remainder

1. Subtract Divisor register from Remainder register, and place the result in Remainder register

Test Remainder

Remainder < 0

Remainder ≥ 0

2a. Shift Quotient register to left, setting new rightmost bit to 1

2b. Restore original value by adding Divisor to Remainder, place sum in Remainder, shift Quotient to the left, setting new least significant bit to 0

3. Shift Divisor register right 1 bit

No: < 33 repetitions

Yes: 33 repetitions

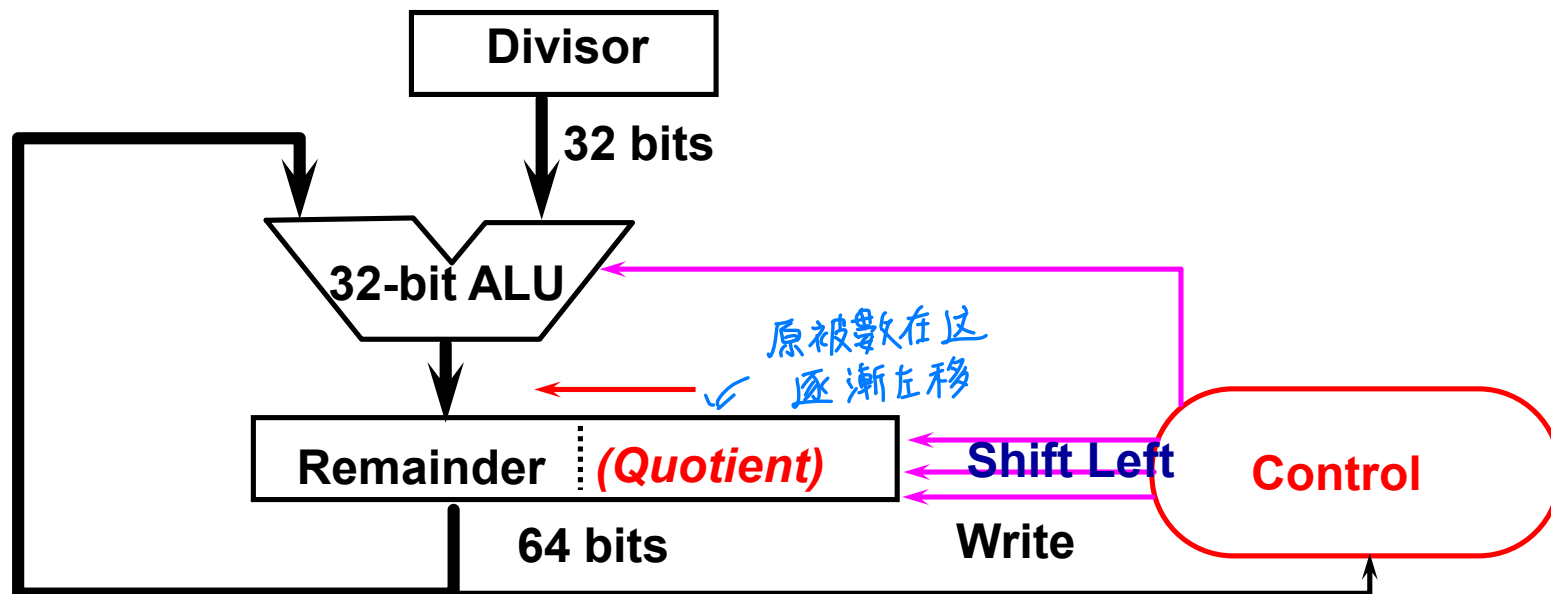
Done

Observations: Divide Version 1

- ◆ Half of the bits in divisor register always 0 → Div Reg 有一半 bit 是 0
=> 1/2 of 64-bit adder is wasted
=> 1/2 of **divisor is wasted**
- ◆ Instead of shifting divisor to right, shift remainder to left? → 除數右移用 remainder 左移取代!
- ◆ 1st step cannot produce a 1 in quotient bit
=> switch order to shift first and then subtract
=> save 1 iteration
- ◆ Eliminate Quotient register by combining with Remainder register as shifted left

Divide Hardware (Version 2)

- ◆ 32-bit Divisor register, 32-bit ALU, 64-bit Remainder register, (0-bit Quotient register)



Divide Algorithm (Version 2)

Start: Place Dividend in Remainder

先把 Remainder 左移一位

1. Shift Remainder register left 1 bit

2. Subtract Divisor register from the left half of Remainder register, and place the result in the left half of Remainder register

Remainder 左半部 - 除数
结果存回 Remainder 左半

Remainder ≥ 0

Test
Remainder

Remainder < 0

0111 / 0010

Step 0 Remainder 0000 Div. 0111 0010

1.1 0000 1110 → 把 Remainder 左移一位

1.2 1110 1110 → 左半 & 除数相减

1.3b 0001 1100 → 塞回左半

2.2 1111 1100 → 还原, 左移!, LSB 设 0!

2.3b 0011 1000

3.2 0001 1000

3.3a 0011 0001

4.2 0001 0001

4.3a 0010 0011

0001 0011

餘

商

R 的左半
右移 1 bit

3a. Shift **Remainder** to left, setting new rightmost bit to 1

3b. Restore original value by adding Divisor to left half of Remainder, and place sum in left half of Remainder. Also shift **Remainder** to left, setting the new least significant bit to 0

→ R-D, 負數!

→ 还原, 左移

→ R-D, 正數!

→ R 左移, LSB 设 1

32nd
repetition?

No: < 32 repetitions

Yes: 32 repetitions

Done. Shift left half of Remainder right 1 bit

Divide

◆ Signed Divides:

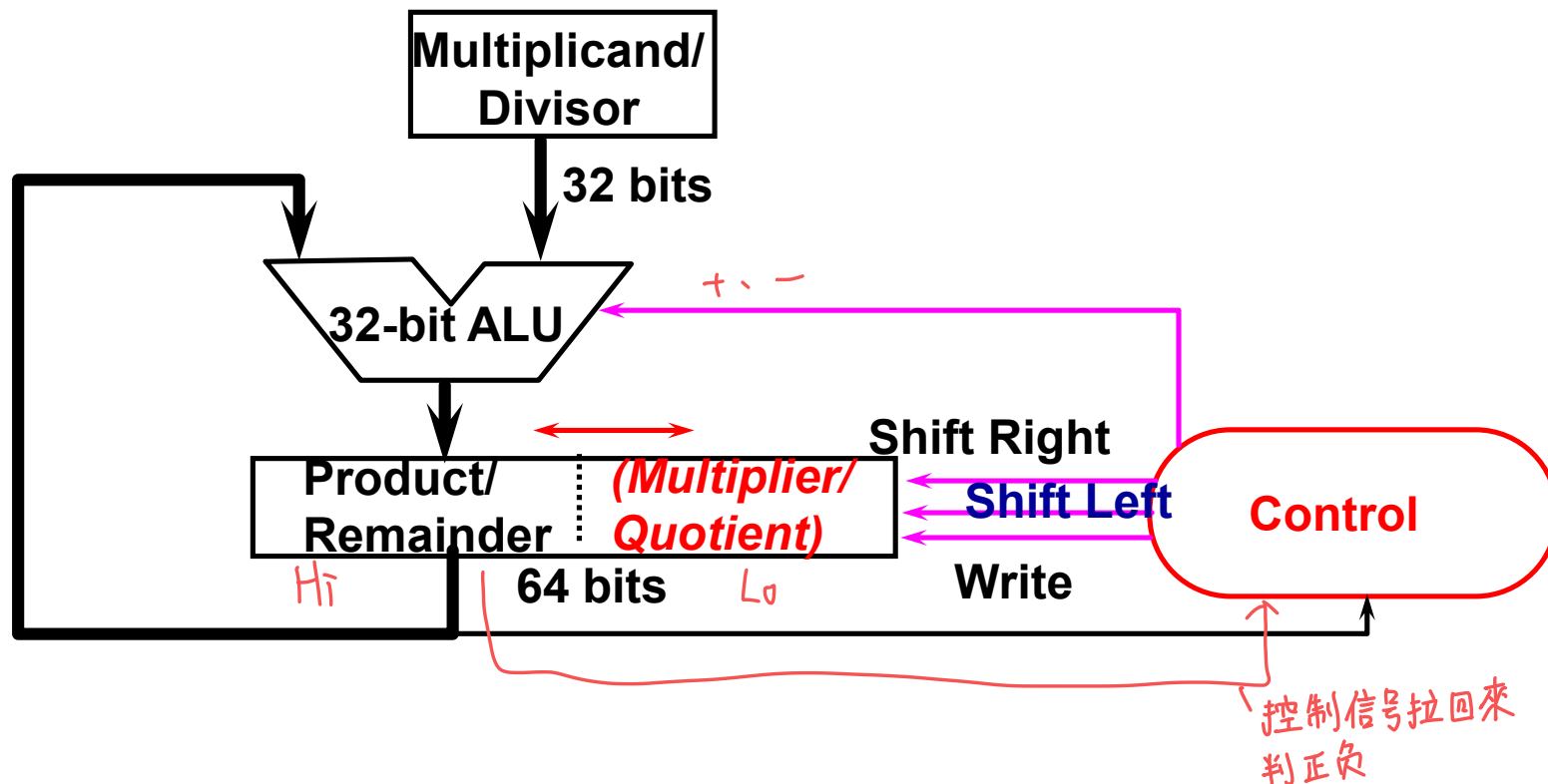
- Remember signs, make positive, complement quotient and remainder if necessary
- Let Dividend and Remainder have same sign and negate Quotient if Divisor sign & Dividend sign disagree, 餘數符号必和商數相同
- e.g., $-7 \div 2 = -3$, remainder = -1
 $-7 \div -2 = 3$, remainder = -1
- Satisfy Dividend = Quotient x Divisor + Remainder

Observations: Multiply and Divide

- ◆ Same hardware as multiply: just need ALU to add or subtract, and 64-bit register to shift left (multiply: shift right)
- ◆ Hi and Lo registers in MIPS combine to act as 64-bit register for multiply and divide

Multiply/Divide Hardware

- ◆ 32-bit Multiplicand/Divisor register, 32-bit ALU, 64-bit Product/Remainder register, (0-bit Multiplier/Quotient register)



Outline

- ◆ Addition and subtraction
- ◆ Constructing an arithmetic logic unit
- ◆ Building ALU
 - Add, sub, and, or, nor
 - Set-on-less-than, overflow detection, zero detection
 - Fast adders
 - Cascaded carry lookahead adder
 - Multiple level carry lookahead adder
- ◆ Multiplication
 - Unsigned multiply
 - Signed multiply
- ◆ Division
- ◆ Floating point
 - Representations
 - Addition and multiplication

Floating-Point: Motivation

◆ What can be represented in N bits?

Assume 16 bit

Unsigned

0 to $2^n - 1$ 0 ~ +65535

☆ 2's Complement

-2^{n-1} to $2^{n-1} - 1$ -32768 ~ +32767

1's Complement

$-2^{n-1} + 1$ to 2^{n-1} -32767 ~ +32767 (有-0和+0!)

☆ Excess M

$-M$ to $2^n - M - 1$

◆ But, what about ...

- very large numbers?

9,349,398,989,787,762,244,859,087,678

- very small number?

0.000000000000000000000000000045691

- rationals

$2/3$ 循環小數

- irrationals

$\sqrt{2}$ 無理數

- transcendentals

e, π 超越數

Assume 32768 當 0

-32768 ~ +32767

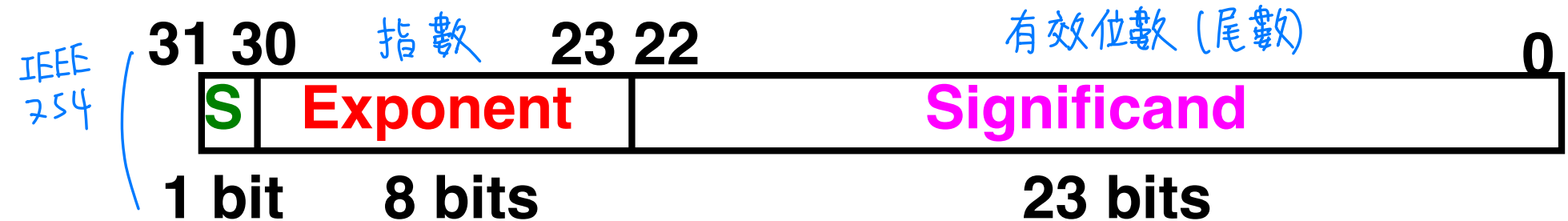
Scientific Notation: Binary



- ◆ Computer arithmetic that supports it is called **floating point**, because the **binary point is not fixed**, as it is for integers
_{小數點不是固定的}
 - ◆ Normalized form: no leading 0s (exactly one digit to left of decimal point) _{→ 正規化: 小數點左邊必為1 (開頭不為0)}
 - ◆ Alternatives to represent 1/1,000,000,000
 - Normalized: 1.0×10^{-9}
 - Not normalized: 0.1×10^{-8} , 10.0×10^{-10}
- e.g.
 $0.111 = 1.11 \times 2^{-1}$
 $11.11 = 1.111 \times 2^1$
省一個 bit 存小數點右邊就好

FP Representation

- ◆ Normal format: $1.\text{xxxxxxxxxx}_{\text{two}} \times 2^{\text{yyyy}_{\text{two}}}$
- ◆ Want to put it into multiple words: 32 bits for *single-precision* and 64 bits for *double-precision* → 放 32 bit : float
64 bit : double
- ◆ A simple single-precision representation:



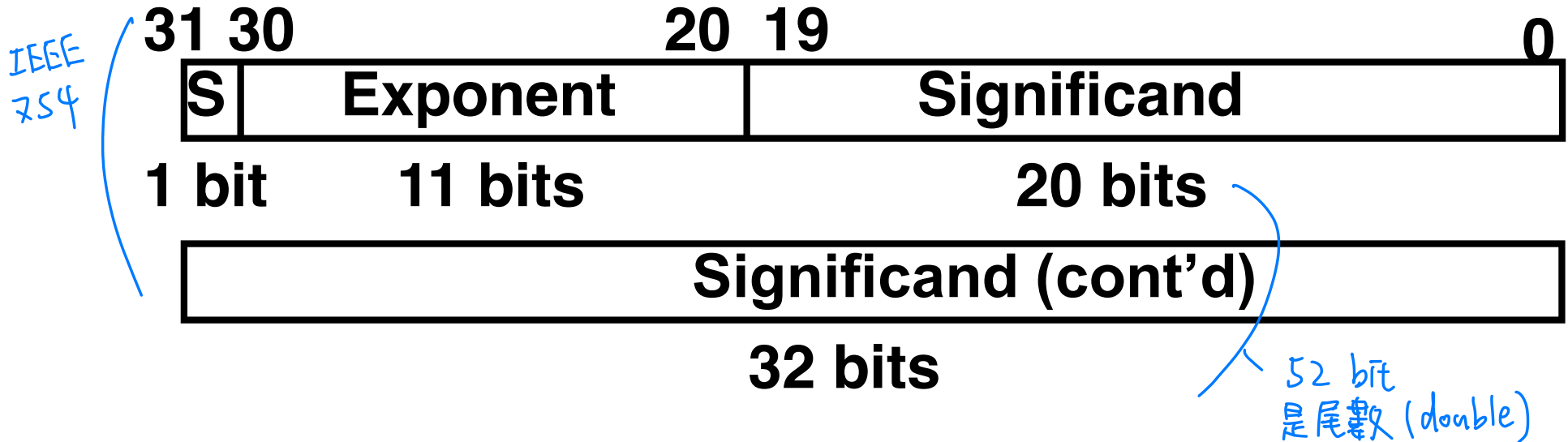
S represents sign

Exponent represents y's

Significand represents x's

Double Precision Representation

- ◆ Next multiple of word size (64 bits)



- ◆ Double precision (vs. single precision)
 - But primary advantage is greater accuracy due to larger significand

IEEE 754 Standard (1/4)

- ◆ Regarding single precision, DP similar
- ◆ Sign bit: (1 bit)
 - 1 means negative
 - 0 means positive
- ◆ Significand:
 - 正規化後把第一個1隱藏了
 - To pack more bits, **leading 1 implicit for normalized numbers**
 - 1 + 23 bits single, 1 + 52 bits double 有效位數 →
 - float : 24 bit
 - double : 53 bit
 - always true: $0 < \text{Significand} < 1$
(for normalized numbers)
- ◆ Note: 0 has no leading 1, so reserve exponent value 0 just for number 0
— 讓exp為0來表示0

IEEE 754 Standard (2/4)

◆ Exponent:

- Need to represent positive and negative exponents
- Also want to compare FP numbers as if they were integers, to help in value comparisons
- If use 2's complement to represent?
e.g., 1.0×2^{-1} versus $1.0 \times 2^{+1}$ (1/2 versus 2)

1/2	0	1111 1111	000 0000 0000 0000 0000 0000
2	0	0000 0001	000 0000 0000 0000 0000 0000

用 2 補數表 exp, 怎麼比大小?

尾數用減的就可以比大小

*If we use integer comparison for these two words,
we will conclude that $1/2 > 2$!!!*

Biased (Excess) Notation

◆ Biased 7

(用7當日)

0000	-7
0001	-6
0010	-5
0011	-4
0100	-3
0101	-2
0110	-1
0111	0
1000	1
1001	2
1010	3
1011	4
1100	5
1101	6
1110	7
1111	8

IEEE 754 Standard (3/4)

- ◆ Instead, let notation 0000 0000 be most negative, and 1111 1111 most positive
- ◆ Called **biased notation**, where bias is the number subtracted to get the real number
 - IEEE 754 uses bias of 127 for single precision: Subtract 127 from Exponent field to get actual value for exponent
 - 1023 is bias for double precision

1.0×2^{-1}
1/2
 1.0×2^1
2

0	0111 1110	000 0000 0000 0000 0000 0000
0	1000 0000	000 0000 0000 0000 0000 0000

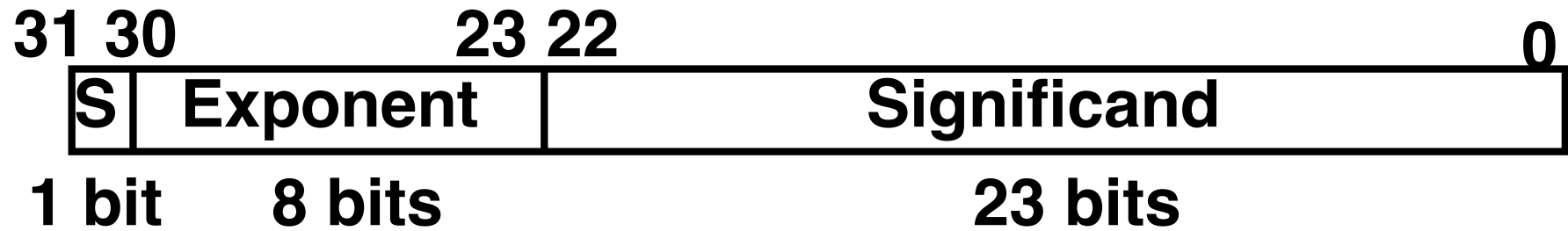
偏移 + exp $\rightarrow 127 + (-1) = 126$

$\rightarrow 127 + (1) = 128$

$\rightarrow$ 相減比大小

IEEE 754 Standard (4/4)

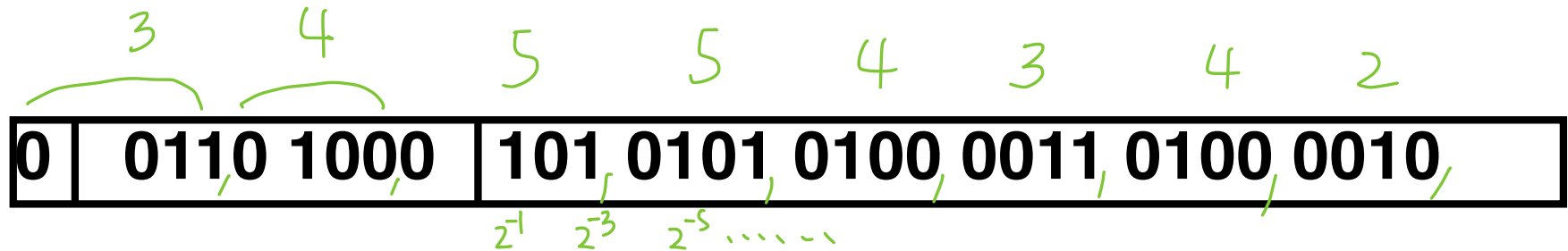
- ◆ Summary (single precision):



$$(-1)^S \times (1.\text{Significand}) \times 2^{(\text{Exponent}-127)}$$

- ◆ Double precision identical, except with exponent bias of 1023

Example: FP to Decimal



◆ Sign: 0 => positive

◆ Exponent:

- $0110\ 1000_{\text{two}} = 104_{\text{ten}}$

- Bias adjustment: $104 - 127 = -23$

◆ Significand:

正規化後 hidden 的

- $1 + 2^{-1} + 2^{-3} + 2^{-5} + 2^{-7} + 2^{-9} + 2^{-14} + 2^{-15} + 2^{-17} + 2^{-22}$
 $= 1.0 + 0.666115$

◆ Represents: $1.666115_{\text{ten}} \times 2^{-23} \approx 1.986 \times 10^{-7}$

Example 1: Decimal to FP

◆ Number = - 0.75

$$= - 0.11_{\text{two}} \times 2^0$$

$$= - 1.1_{\text{two}} \times 2^{-1}$$

$$\begin{array}{r} 0.75 \\ \times 2 \\ \hline 1.5 \end{array} \quad \begin{array}{r} 0.5 \\ \times 2 \\ \hline 1.0 \end{array}$$

(base 2 scientific notation)

(normalized scientific notation)

◆ Sign: negative => 1

◆ Exponent:

● Bias adjustment: $-1 + 127 = 126$

● $126_{\text{ten}} = 0111\ 1110_{\text{two}}$



Fraction Part: Decimal to Binary

♦ $0.75 \times 2 = 1.5$

♦ $0.5 \times 2 = 1.0$

0.11



Example 2: Decimal to FP

◆ A more difficult case: representing $1/3$?

$= 0.33333..._{10} = 0.0101010101..._2 \times 2^0$ (base 2) ↗ 循環小數
 $= 1.0101010101..._2 \times 2^{-2}$ ↖ 一直×2 會得一個循環2進位
(normalization) -2 + 127 = 125

- Sign: 0 → 正數
- Exponent = $-2 + 127 = 125_{10} = 01111101_2$
- Significand = 0101010101...



Single-Precision Range

- ◆ Exponents 00000000 and 11111111 reserved
- ◆ Smallest value
 - Exponent: 00000001 (-126)
 $\Rightarrow$ actual exponent = $1 - 127 = -126$
 - Fraction: 000...00 $\Rightarrow$ significand = 1.0 $\rightarrow$ 小數點後都0 $\rightarrow 1.0$
 - $\pm 1.0 \times 2^{-126} \approx \pm 1.2 \times 10^{-38} \rightarrow$ 最小值
- ◆ Largest value
 - exponent: 11111110 (-254)
 $\Rightarrow$ actual exponent = $254 - 127 = +127$
 - Fraction: 111...11 $\Rightarrow$ significand $\approx 10.2 = 2.0$ $\rightarrow$ 小數點後都1
 $\downarrow$
1.111111 ≈ 2
 - $\pm 2.0 \times 2^{+127} \approx \pm 3.4 \times 10^{+38} \rightarrow$ 最大值

Double-Precision Range

- ◆ Exponents 0000...00 and 1111...11 reserved
- ◆ Smallest value
 - Exponent: 000000000001
 $\Rightarrow$ actual exponent = $1 - 1023 = -1022$
 - Fraction: 000...00 $\Rightarrow$ significand = 1.0
 - $\pm 1.0 \times 2^{-1022} \approx \pm 2.2 \times 10^{-308}$ 最小
- ◆ Largest value
 - Exponent: 111111111110
 $\Rightarrow$ actual exponent = $2046 - 1023 = +1023$
 - Fraction: 111...11 $\Rightarrow$ significand $\approx 10.2 = 2.0$
 - $\pm 2.0 \times 2^{+1023} \approx \pm 1.8 \times 10^{+308}$ $\rightarrow$ 最大

Floating-Point Precision

◆ Relative precision

- all fraction bits are significant

- Single: approx 2^{-23}

- Equivalent to $23 \times \log_{10} 2 \approx 23 \times 0.3 \approx 6$ decimal digits of precision

single 的精度可以到小數點後6位

- Double: approx 2^{-52}

- Equivalent to $52 \times \log_{10} 2 \approx 52 \times 0.3 \approx 16$ decimal digits of precision

Zero and Special Numbers

- ◆ What have we defined so far? (single precision)

<u>Exponent</u>	<u>Significand</u>	<u>Object</u>
0	0	<u>???</u>
0	nonzero	<u>???</u>
1-254	anything	+/- floating-point
255	0	<u>???</u>
255	nonzero	<u>???</u>

Zero and Special Numbers

- ◆ What have we defined so far? (single precision)

<u>Exponent</u>	<u>Significand</u>	<u>Object</u>
<u>0</u>	<u>0</u>	<u>???</u>
<u>0</u>	<u>nonzero</u>	<u>???</u>
1-254	anything	+/- floating-point
<u>255</u>	<u>0</u>	<u>???</u>
<u>255</u>	<u>nonzero</u>	<u>???</u>

Representation for 0

◆ Represent 0?

一指、尾都0來表0

- **exponent all zeroes**
 - **significand all zeroes too**
 - **What about sign?**
 - **+0:** 0 00000000 00000000000000000000000000000000
 - **-0:** 1 00000000 00000000000000000000000000000000
- Sign Bit!*

Special Numbers

- ◆ What have we defined so far? (single precision)

<u>Exponent</u>	<u>Significand</u>	<u>Object</u>
0	0	0
0	nonzero	???
1-254	anything	+/- floating-point
255	0	???
255	nonzero	???

- ◆ Range:

$$1.0 \times 2^{-126} \approx 1.8 \times 10^{-38}$$

What if result too small? (>0 , $< 1.8 \times 10^{-38} \Rightarrow$ Underflow!)

$$(2 - 2^{-23}) \times 2^{127} \approx 3.4 \times 10^{38}$$

What if result too large? ($> 3.4 \times 10^{38} \Rightarrow$ Overflow!)

Gradual Underflow

非正規化的數字

- ◆ Represent denormalized numbers (denorms)

- Exponent : all zeroes 指是 0
- Significand : non-zeroes → 尾不是 0 → Gradual underflow

0	0000 0000	0100 0000 0000 0000 0000 000
---	-----------	------------------------------

$= 0.01_2 \times 2^{-126}$ → 表一个非正規化的數字

- Allow a number to degrade in significance until it become 0 (gradual underflow)

Smallest Number

- The smallest **normalized** number 正規化最小可表
 - $1.0000\ 0000\ 0000\ 0000\ 0000\ 000 \times 2^{-126}$
- The smallest **de-normalized** number 非正規化最小可表
 - $0.0000\ 0000\ 0000\ 0000\ 0000\ 001 \times 2^{-126}$

Special Numbers

- ◆ What have we defined so far? (single precision)

Exponent

0

0

1-254

255

255

Significand

0

nonzero

anything

0

nonzero

Object

0

denorm (Gradual underflow)

+/- floating-point

???

???

Representation for +/- Infinity

— $\div 0$ 會產生 $+/-$ 無限大, 而非 overflow

- ◆ In FP, divide by zero should produce +/- infinity, not overflow
- ◆ Why?
 - OK to do further computations with infinity, e.g., $X/0 > Y$ may be a valid comparison
- ◆ IEEE 754 represents +/- infinity
 - Most positive exponent reserved for infinity
 - Significands all zeroes

可以 + 或 -

指 | 尾 0 表無限大

S	1111 1111	0000 0000 0000 0000 0000 000
---	-----------	------------------------------

Special Numbers (cont'd)

- ◆ What have we defined so far? (single-precision)

<u>Exponent</u>	<u>Significand</u>	<u>Object</u>
0	0	0
0	nonzero	denom
1-254	anything	+/- fl. pt. #
255	0	+/- infinity
255	nonzero	<u>???</u>

Representation for Not a Number

- ◆ What do I get if I calculate $\text{sqrt}(-4.0)$ or $0/0$?
 - If infinity is not an error, these should not be either
 - They are called *Not a Number* (NaN)
 - Exponent = 255, Significand nonzero
- ◆ Why is this useful?
 - Hope NaNs help with debugging?
 - They contaminate: $\text{op}(\text{NaN}, X) = \text{NaN}$ → 任意數和NaN計算都是NaN
 - OK if calculate but don't use it

Special Numbers (cont'd)

- ◆ What have we defined so far? (single-precision)

<u>Exponent</u>	<u>Significand</u>	<u>Object</u>
0	0	0
0	nonzero	denom
1-254	anything	+/- fl. pt. #
255	0	+/- infinity
255	nonzero	NaN

Outline

- ◆ Addition and subtraction
- ◆ Constructing an arithmetic logic unit
 - Building ALU
 - Add, sub, and, or, nor
 - Set-on-less-than, overflow detection, zero detection
 - Fast adders
 - Cascaded carry lookahead adder
 - Multiple level carry lookahead adder
- ◆ Multiplication
 - Unsigned multiply
 - Signed multiply
- ◆ Division
- ◆ Floating point
 - Representations
 - Addition and multiplication

Floating-Point Addition

Basic addition algorithm:

Assume: $\frac{X_m}{\text{尾}} \times 2^{\frac{X_e}{\text{指}}} + \frac{Y_m}{\text{尾}} \times 2^{\frac{Y_e}{\text{指}}}$

(1) Align binary point : compute $Y_e - X_e$

- ◆ **right shift the smaller number**, say X_m , that many positions to form $X_m \times 2^{X_e - Y_e}$

(2) Add mantissa: compute $X_m \times 2^{X_e - Y_e} + Y_m$

→ 假设加完後有進位
e.g. 10.1101×2^5
↳ 還要正規化一次!
 1.01101×2^6

(3) **Normalization & check for over/underflow if necessary:**

- left shift result, decrement result exponent
- right shift result, increment result exponent
- check overflow or underflow during the shift

檢查是否要重新
→ 正規化 & +/-

→ 四捨五入後可能還要做一次正規化!

(4) Round the mantissa and renormalize if necessary

e.g. $X \ 1.001 \times 2^5$
 $Y \ 1.1001 \times 2^2$
↳ 0.001001×2^5

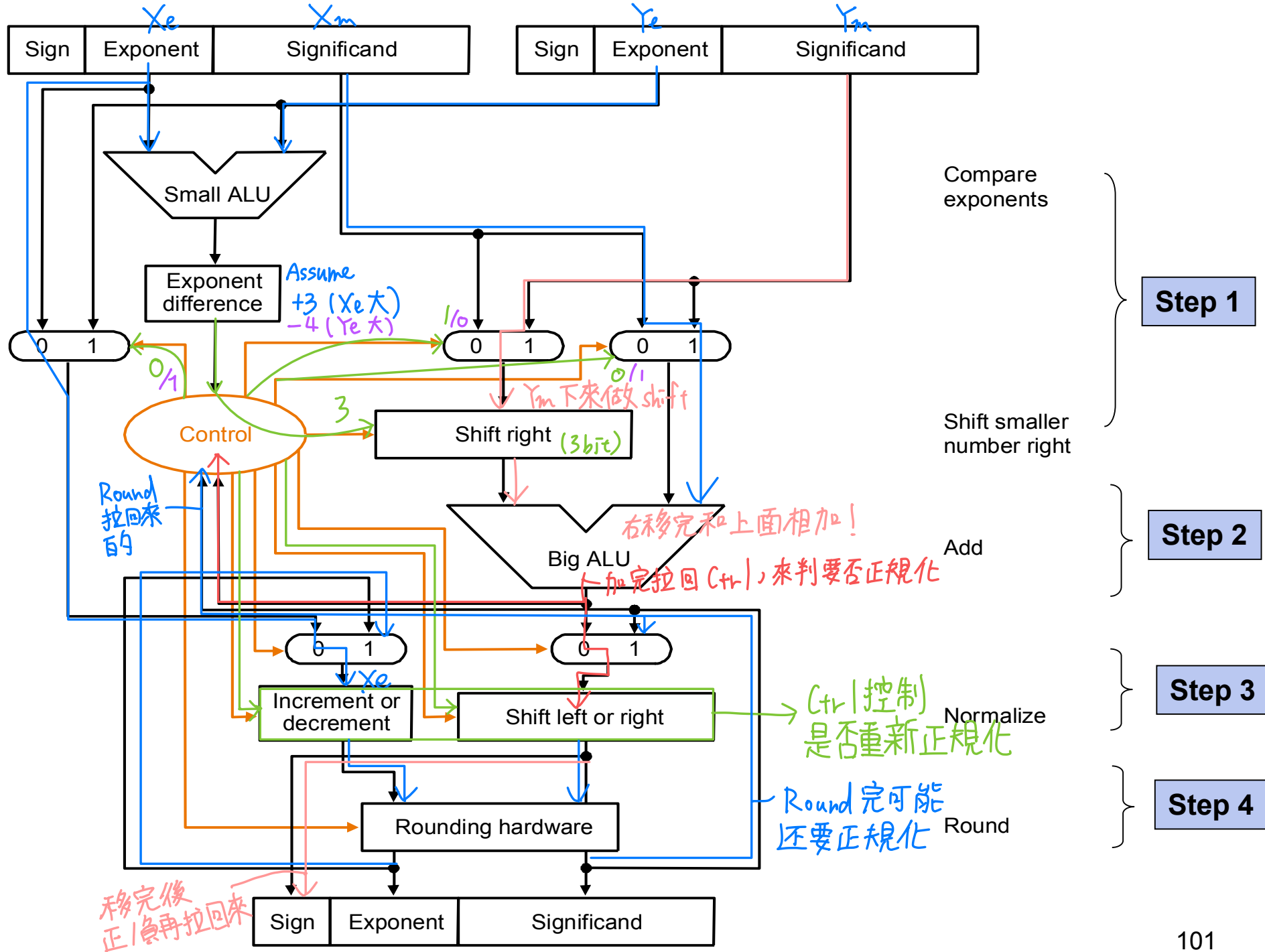
$5 - 2 = +3$
↳ X 比 Y 大, Y 要右移 3 bit

$$\begin{array}{r} 1.001000 \\ + 0.001001 \\ \hline 1.010001 \end{array} \times 2^5 \rightarrow \text{完成!}$$

Floating-Point Addition Example

$$\begin{aligned} &1.110 \times 2^{-2} \\ \rightarrow &0.0111 \times 2^0 \\ \rightarrow &0.4375_{10} \end{aligned}$$

- ◆ Now consider a 4-digit binary example
 - $1.000_2 \times 2^{-1} + -1.110_2 \times 2^{-2}$ (0.5 + -0.4375)
- ◆ 1. **Align binary points** → 小的指对齊!
 - Shift number with smaller exponent
 - $1.000_2 \times 2^{-1} + -0.111_2 \times 2^{-1}$
- ◆ 2. Add mantissa
 - $1.000_2 \times 2^{-1} + -0.111_2 \times 2^{-1} = 0.001_2 \times 2^{-1}$
- ◆ 3. Normalize result & check for over/underflow
 - $1.000_2 \times 2^{-4}$, with no over/underflow
- ◆ 4. Round and renormalize if necessary
 - $1.000_2 \times 2^{-4}$ (no change) = 0.0625



Example: $0.5 + (-0.4375)$

$$0.5 = 1.000_2 \times 2^{-1}$$

$$= 0\mathbf{01111110}00000\dots00_2$$

$$-0.4375 = -1.110_2 \times 2^{-2}$$

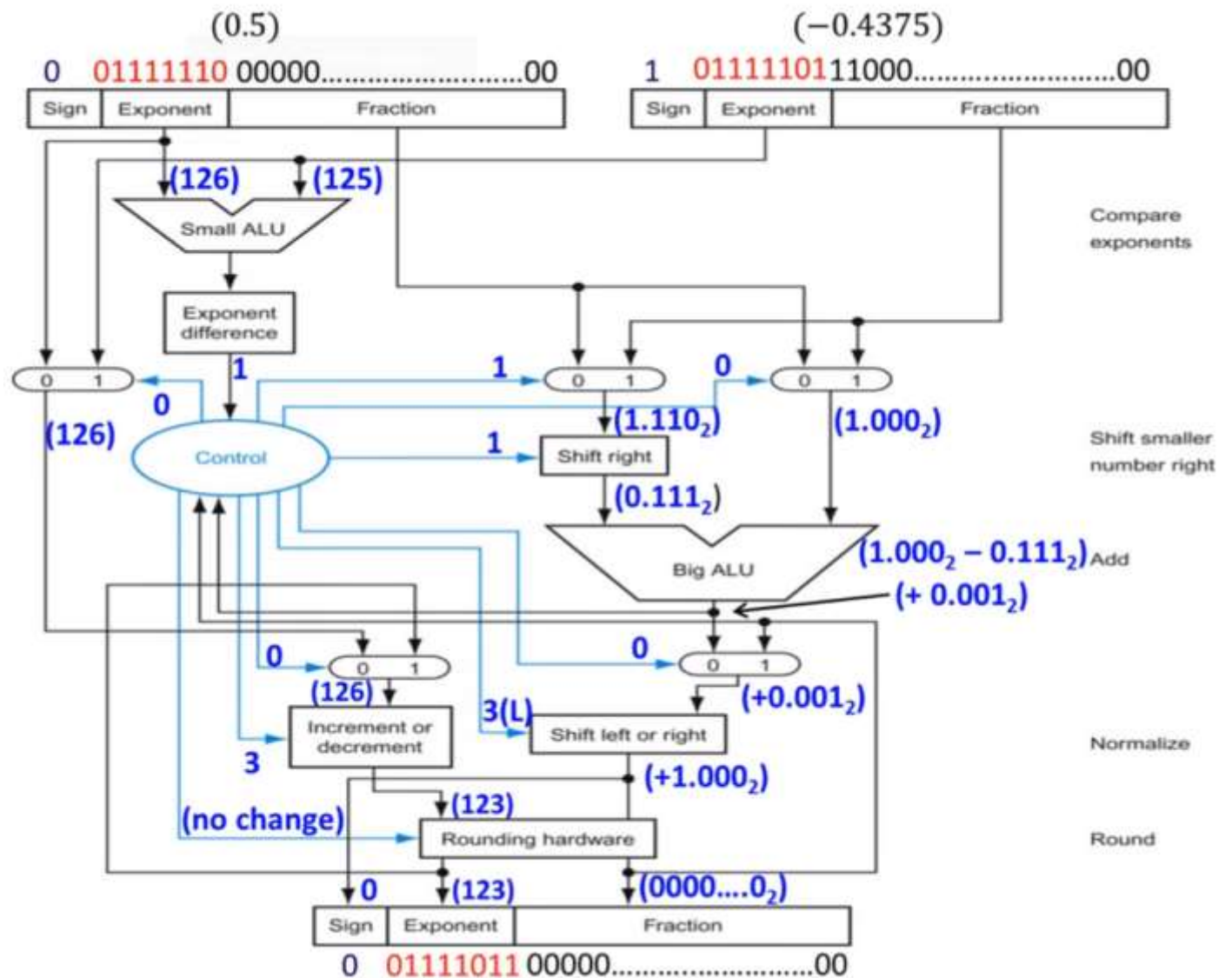
$$= 1\mathbf{01111101}11000\dots00_2$$

Step 1: $-1.110_2 \times 2^{-2} = -0.111_2 \times 2^{-1}$

Step 2: $1.000_2 \times 2^{-1} + (-0.111_2 \times 2^{-1})$
 $= 0.001_2 \times 2^{-1}$

Step 3: $0.001_2 \times 2^{-1} = 1.000_2 \times 2^{-4}$

Step 4: $1.000_2 \times 2^{-4}$
 $= 0\mathbf{01111011}00000\dots00_2$



FP Adder Hardware

- ◆ Much more complex than integer adder
- ◆ Doing it in one clock cycle would take too long

- Much longer than integer operations

↳ 若要在 1 clock 內做完
會花很長時間

- Slower clock would penalize all instructions

- ◆ FP adder usually takes several cycles

- Can be pipelined → FP的運算可以 pipelined

FP 運算時間遠大於 int 運算

clock 拉太長又會處罰到其他指令 (讓其他指令變慢)

Floating-Point Multiplication

Assume
 $X_m \times 2^{X_e} \times Y_m \times 2^{Y_e}$

Basic multiplication algorithm

(1) Add exponents of operands to get exponent of product

doubly biased exponent must be corrected:

$X_e = 7$ $X_e = 1111 = 15 = 7 + 8$

$Y_e = -3$ $Y_e = 0101 = 5 = -3 + 8$

Excess 8 (8 當 0) $10100 = 20 = 4 + 8 + 8$

$= 4 + 8 = 12 = 01100$ 錯了! 8+了2次, 還要減一個 Bias

need extra subtraction step of the bias amount

(2) Multiplication of operand mantissa

(3) Normalize the product & check overflow or underflow during the shift

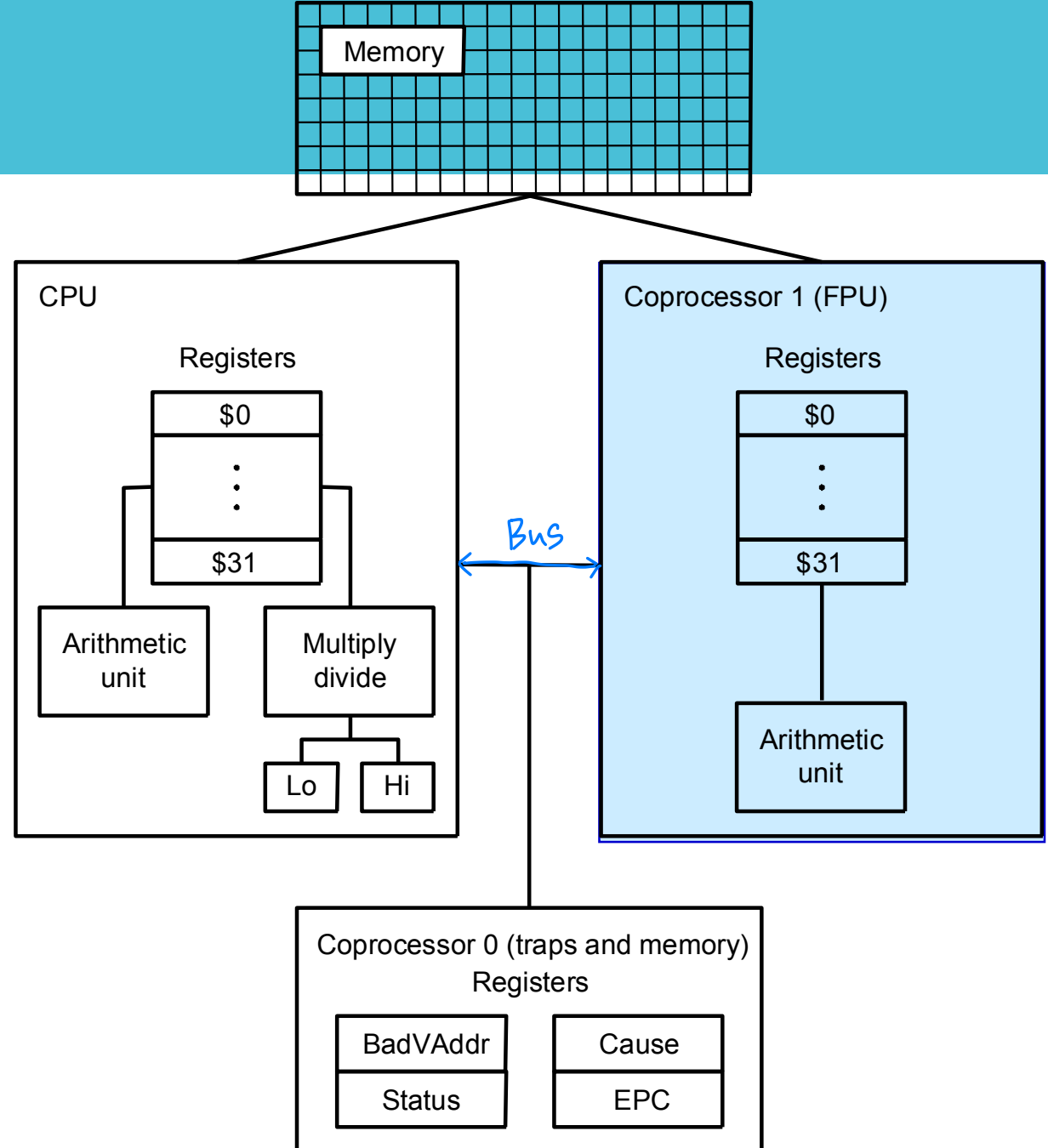
(4) Round the mantissa and renormalize if necessary

(5) Set the sign of product

Floating-Point Multiplication Example

- ◆ Now consider a 4-digit binary example
 - $1.000_2 \times 2^{-1} \times -1.110_2 \times 2^{-2}$ (0.5×-0.4375)
- ◆ 1. Add exponents 指數相加!
 - Unbiased: $-1 + -2 = -3$
 - Biased: $(-1 + 127) + (-2 + 127) = -3 + 254 - 127 = -3 + 127$ ¹²⁴
- ◆ 2. Multiply operand mantissa 尾數相乘
 - $1.000_2 \times 1.110_2 = 1.110_2 \Rightarrow 1.110_2 \times 2^{-3}$
- ◆ 3. Normalize result & check for over/underflow
 - $1.110_2 \times 2^{-3}$ (no change) with no over/underflow → 不用重做正規化
- ◆ 4. Round and renormalize if necessary
 - $1.110_2 \times 2^{-3}$ (no change) → 不用 round
- ◆ 5. Determine sign: → 決定符号!
 - $-1.110_2 \times 2^{-3} = -0.21875$

MIPS R2000 Organization



MIPS Floating Point

- ◆ Separate floating point instructions: 浮点指令都有特別標示
 - Single precision: add.s, sub.s, mul.s, div.s
 - Double precision: add.d, sub.d, mul.d, div.d
- ◆ FP part of the processor: 浮点有特別的 Reg
 - contains 32 32-bit registers: \$f0, \$f1, ...
 - most registers specified in .s and .d instruction refer to this set 一倍精会拿2个 Reg 組成一个 64 bit
 - Double precision: by convention, even/odd pair contain one DP FP number: \$f0/\$f1, \$f2/\$f3
 - separate load and store: lwc1 and swc1 → 浮点的 load 和 store
 - Instructions to move data between main processor and coprocessors:
 - mfc0, mtc0, mfc1, mtc1, etc.

Associativity

結合律存在? (e.g. $(a+b)+c = a+(b+c)$)

◆ Floating Point add, subtract associative ?

		$(x+y)+z$	$x+(y+z)$
x	-1.50E+38	0.00E+00 ✓ result = 0	-1.50E+38
y	1.50E+38		1.50E+38 ↓ !!!
z	1.0	1.0 0+1.0 ↓	
		1.00E+00	0.00E+00

z 是很大的數 (1.5×10^{38})
而 x 很小。
所以 $x+z$ 完之後, z 就
不見了!

浮點在加/減是沒有結合律的

◆ Therefore, Floating Point add, subtract are not associative!

- Why? FP result approximates real result!
- This example: 1.5×10^{38} is so much larger than 1.0 that $1.5 \times 10^{38} + 1.0$ in floating point representation is still 1.5×10^{38}